

**Developers** wanting access to these APIs can make requests through the Developer Portal, which can either immediately provide access or require approval by the publisher/administrator. Developers can also view the API documentation and sample code in the Developer Portal, to rapidly adopt the APIs offered by the web service.

The **apps** that these developers create then access the API through the proxy offered by Azure API Management. The proxy both provides a layer of obscurity, hiding the actual end-point of the API on the publisher/administrator's server and can also include additional logic like API translation to ensure the exposed API is kept consistent when a call to one API is redirected to another. It can also use IP filtering to block API calls originating from a specific IP domain or set of domains. Azure API Management also keeps its web services secure by using a set of public keys, called API keys, to authenticate and authorize each API call. When authorization fails, access to the API and the functionality it supports is blocked.

Azure API Management can also reduce the number of API calls to a service (a procedure called throttling) to optimize the performance of the web service. To learn more, review [Azure API Management](#) and Azure API Management at AzureCon 2015.

## 4 Data-at-rest security methods

When data arrives on a device, we refer to it as "data-at-rest." This data needs to be stored on the device in a secure manner, so that it cannot be accessed by unauthorized users or apps. The app model in Windows does a lot to ensure that the data stored by any app is only accessible to that app, while providing APIs to share the data when necessary. Additional APIs are also available to ensure that data can be encrypted and credentials can be stored safely.

### 4.1 Windows app model

Traditionally, Windows has never had a definition of an app. It was most commonly referred to as an executable (.exe), and this never included installation, storage of state, execution length, versioning, OS integration, or app-to-app communication. The Universal Windows Platform model defines an app model that covers installation, runtime environment, resource management, updates, data model, and uninstallation.

Windows 10 apps run in a container, which means that they have limited privileges by default (additional privileges can be requested and granted by the user). For example, if an app wants to access files on the system, a file picker from the [Windows.Storage.Pickers](#) namespace has to be used to let the user pick a file (no direct access to files is enabled). Another example is if an app wants to access the user's

location data, it needs to enable the location device capability needs to be declared, prompting the user at download time that this app will request access to the user's location. On top of that, the first time the app wants to access the user's location, an additional consent prompt is shown to the user, requesting permission to access the data.

Note that this app model acts as a "jail" for apps, meaning that they can't reach out, but it is not a "castle" that cannot be reached from the outside (applications with administrator privileges can of course still reach in). Device Guard in Windows, which enables organizations/IT to specify which (Win32) apps are allowed to execute, can further help limit this access.

The app model also manages the app lifecycle. It limits the background execution of apps by default, for example; as soon as an app goes into the background, the process is suspended – after giving the app a brief period to address app suspension in code – and its memory is frozen. The operating system does provide mechanisms for apps to ask for specific background task execution (on a schedule, triggered by various events such as Internet/Bluetooth connectivity, power changes, etc., and in specific scenarios such as music playing or GPS tracking).

When memory resources on the device are running low, Windows frees memory space by terminating apps. This lifecycle model forces apps to persist data whenever they're suspended, because there is no additional time available between suspension and termination.

For more information, see [It's Universal: Understanding the Lifecycle of a Windows 10/11 Application](#) .

## 4.2 Stored credential protection

Windows apps that access authenticated services often provide the users the option of storing their credentials on the local device. This is a convenience for the users; when they provide their username and password, the app automatically uses them in subsequent launches of the app. Because this can be a security issue if an attacker gains access to this stored data, Windows provides the ability for packaged apps to store user credentials in a secure credential locker. The app calls the Credential Locker API to store and retrieve the credentials from the locker instead of storing them in the app's storage container. The credential locker is managed by the operating system, but access is limited to the app that stores them, providing a securely managed solution for credential storage.

When a user supplies the credentials to be stored, the app gets a reference to the credential locker using the [PasswordVault](#) object in the [Windows.Security.Credentials](#) namespace. It then creates a [PasswordCredential](#) object containing an identifier for the Windows app and the username and password. This is passed to the [PasswordVault.Add](#) method to store the credentials in the locker. The following C# code example shows how this is done.

CS

```
var vault = new PasswordVault();
vault.Add(new PasswordCredential("My App", username, password));
```

In the following C# code example, the app requests all of the credentials corresponding to the app by calling the [FindAllByResource](#) method of the [PasswordVault](#) object. If more than one is returned, it prompts the user to enter their username. If the credentials are not in the locker, the app prompts the user for them. The user is then logged into the server using the credentials.

CS

```
private string resourceName = "My App";
private string defaultUserName;

private void Login()
{
    PasswordCredential loginCredential = GetCredentialFromLocker();

    if (loginCredential != null)
    {
        // There is a credential stored in the locker.
        // Populate the Password property of the credential
        // for automatic login.
        loginCredential.RetrievePassword();
    }
    else
    {
        // There is no credential stored in the locker.
        // Display UI to get user credentials.
        loginCredential = GetLoginCredentialUI();
    }
    // Log the user in.
    ServerLogin(loginCredential.UserName, loginCredential.Password);
}

private PasswordCredential GetCredentialFromLocker()
{
    PasswordCredential credential = null;

    var vault = new PasswordVault();
```

```

        IReadOnlyList<PasswordCredential> credentialList = null;

        try
        {
            credentialList = vault.FindAllByResource(resourceName);
        }
        catch(Exception)
        {
            return null;
        }

        if (credentialList.Count == 1)
        {
            credential = credentialList[0];
        }
        else if (credentialList.Count > 0)
        {
            // When there are multiple usernames,
            // retrieve the default username. If one doesn't
            // exist, then display UI to have the user select
            // a default username.
            defaultUserName = GetDefaultUserNameUI();

            credential = vault.Retrieve(resourceName, defaultUserName);
        }
        return credential;
    }
}

```

For more information, see [Credential locker](#).

## 4.3 Stored data protection

When you are dealing with stored data, commonly referred to as data-at-rest, encrypting it can prevent unauthorized users from accessing the stored data. The two common mechanisms to encrypt data are using either symmetric keys or using asymmetric keys. However, data encryption can't ensure that the data is unaltered between the time it was sent and the time it was stored. In other words, the data integrity cannot be ensured. Using message authentication codes, hashes, and digital signing are common techniques to solve this problem.

### 4.3.1 Data encryption

With symmetric encryption, both the sender and recipient have the same key and use it to both encrypt and decrypt the data. The challenge with this approach is securely sharing the key so both parties are aware of it.

One answer to this is asymmetric encryption, in which a public/private key pair is used. The public key is shared freely with anyone who wants to encrypt a message. The private key is always kept secret so that only you can use it to decrypt the data. A common technique to allow for discovery of the public key is by using digital certificates, also simply referred to as certificates. The certificate holds information about the public key, in addition to information about the user or server such as the name, issuer, email address and country.

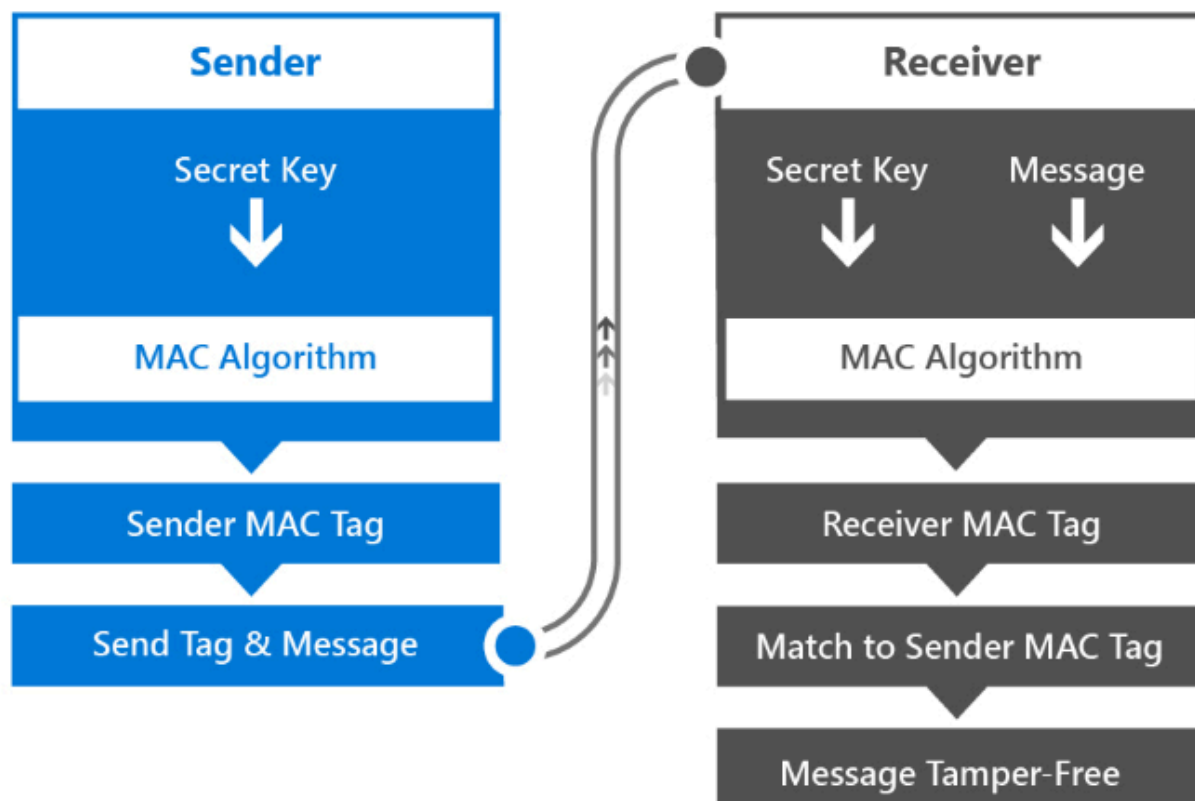
Windows app developers can use the [SymmetricKeyAlgorithmProvider](#) and [AsymmetricKeyAlgorithmProvider](#) classes to implement symmetric and asymmetric encryption in their UWP apps. Additionally, the [CryptographicEngine](#) class can be used to encrypt and decrypt data, sign content and verify digital signatures. Apps can also use the [DataProtectionProvider](#) class in the [Windows.Security.Cryptography.DataProtection](#) namespace to encrypt and decrypt stored local data.

## 4.3.2 Detecting message tampering (MACs, hashes, and signatures)

A MAC is a code (or tag) that results from using a symmetric key (called the secret key) or a message as input to a MAC encryption algorithm. The secret key and the algorithm are agreed upon by the sender and receiver before the message transfer.

MACs verify messages like this.

- The sender derives the MAC tag by using the secret key as input to the MAC algorithm.
- The sender sends the MAC tag and the message to the receiver.
- The receiver derives the MAC tag by using the secret key and the message as inputs to the MAC algorithm.
- The receiver compares their MAC tag with the sender's MAC tag. If they are the same then we know that the message has not been tampered with.



Windows apps can implement MAC message verification by calling the [MacAlgorithmProvider](#) class to generate the key and [CryptographicEngine](#) class to perform the MAC encryption algorithm.

### 4.3.3 Using hashes

A hash function is a cryptographic algorithm that takes an arbitrarily long block of data and returns a fixed-size bit string called a hash value. There is an entire family of hash functions that can do this.

A hash value can be used in place of a MAC in the message-transfer scenario above. The sender sends a hash value and a message, and the receiver derives their own hash value from the sender's hash value and message and compares the two hash values. Apps running on Windows can call the [HashAlgorithmProvider](#) class to enumerate the hash algorithms that are available and run one of them. The [CryptographicHash](#) class represents the hash value. The [CryptographicHash.GetValueAndReset](#) method can be used to repeatedly hash different data without having to re-create the object for each use. The Append method of the [CryptographicHash](#) class adds new data to a buffer to be hashed. This entire process is shown in the following C# code example.

```

public void SampleReusableHash()
{
    // Create a string that contains the name of the
    // hashing algorithm to use.
    string strAlgName = HashAlgorithmNames.Sha512;

    // Create a HashAlgorithmProvider object.
    HashAlgorithmProvider objAlgProv =
    HashAlgorithmProvider.OpenAlgorithm(strAlgName);

    // Create a CryptographicHash object. This object can be reused to
    continually
    // hash new messages.
    CryptographicHash objHash = objAlgProv.CreateHash();

    // Hash message 1.
    string strMsg1 = "This is message 1";
    IBuffer buffMsg1 = CryptographicBuffer.ConvertStringToBinary(strMsg1,
    BinaryStringEncoding.Utf16BE);
    objHash.Append(buffMsg1);
    IBuffer buffHash1 = objHash.GetValueAndReset();

    // Hash message 2.
    string strMsg2 = "This is message 2";
    IBuffer buffMsg2 = CryptographicBuffer.ConvertStringToBinary(strMsg2,
    BinaryStringEncoding.Utf16BE);
    objHash.Append(buffMsg2);
    IBuffer buffHash2 = objHash.GetValueAndReset();

    // Convert the hashes to string values (for display);
    string strHash1 = CryptographicBuffer.EncodeToBase64String(buffHash1);
    string strHash2 = CryptographicBuffer.EncodeToBase64String(buffHash2);
}

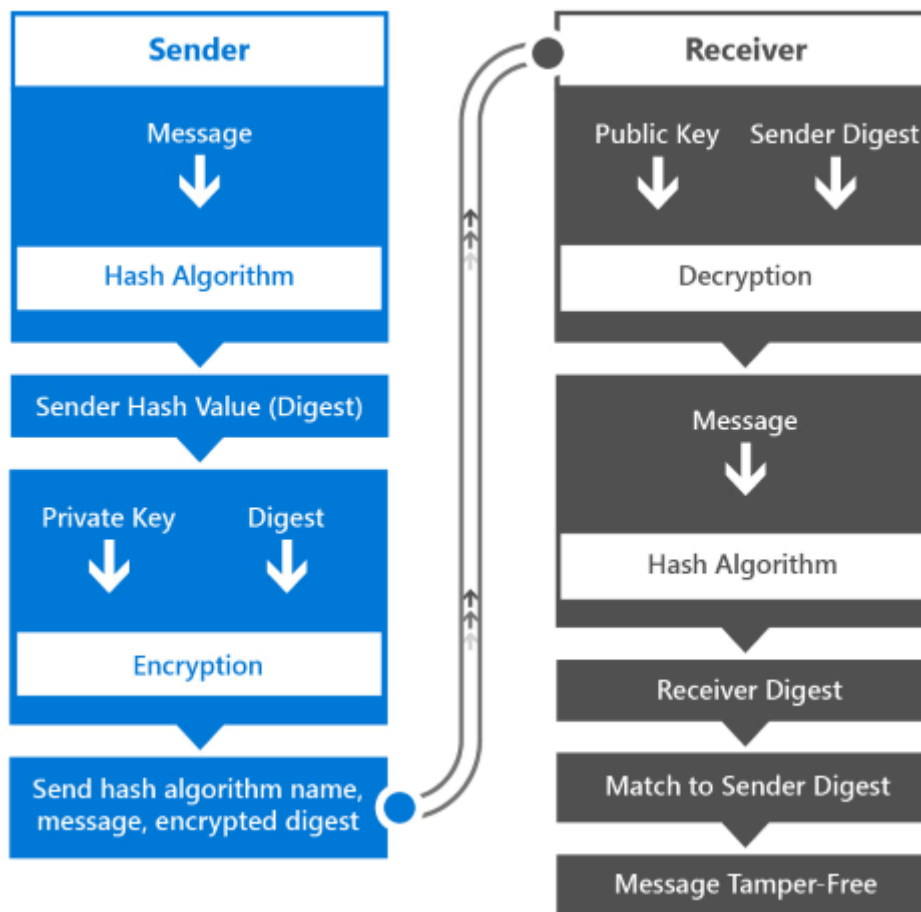
```

## 4.3.4 Digital signatures

The data integrity of a digitally signed stored message is verified in a similar way to MAC authentication. Here is the way the digital signature workflow operates.

- The sender derives a hash value (also known as a digest) by using the message as the input to a hash algorithm.
- The sender encrypts the digest using their private key.
- The sender sends the message, the encrypted digest, and the name of the hash algorithm that was used.
- The receiver uses the public key to decrypt the encrypted digest it received. It then uses the hash algorithm to hash the message to create a digest of its own. And finally the receiver compares the two digests (the one it received and decrypted, and the one it made). Only if the two match can the receiver be sure that the

message was sent by the possessor of the private key, and therefore they are who they say they are, and that the message was not altered in transit.



Hashing algorithms are very fast, so hash values can be derived quickly from even large messages. The resulting hash value is an arbitrary length and can be shorter than the full message, so using public and private keys to encrypt and decrypt only the digest rather than the full message is an optimization.

For more information, take a look at articles on [Digital signatures](#), [MACs](#), [hashes](#), and [signatures](#), and [Cryptography](#).

## 5 Summary

The Universal Windows Platform in Windows offers a number of ways to leverage operating system capabilities to create more secure apps. In different authentication scenarios, such as single-factor, multi-factor, or brokered authentication with an OAuth identity provider, APIs exist to mitigate the most common challenges with authentication. Windows Hello provides a new biometric sign-in system that recognizes the user and actively defeats efforts to circumvent proper identification. It also delivers multiple layers of keys and certificates that can never be revealed or used outside the

trusted platform module. Plus, a further layer of security is available through the optional use of attestation identity keys and certificates.

To secure data in flight, APIs exist to communicate with remote systems securely over SSL, while providing the possibility to validate the server's authenticity with SSL pinning. Publishing APIs securely and in a controlled manner is something in which Azure API Management aids by providing powerful configuration options for exposing APIs across the web using a proxy that provides additional obfuscation of the API endpoint. Access to these APIs is secured by using API keys and API calls can be throttled to control performance.

When the data arrives on the device, the Windows app model provides more control over how the app is installed, updated and accesses its data, while keeping it from accessing data of other apps in an unauthorized manner. Credential locker can provide secure storage of user credentials that is managed by the operating system and other data can be protected on the device by using the encryption and hashing APIs offered by the Universal Windows Platform.

## 6 Resources

### 6.1 How-to articles

- [Authentication and user identity](#)
- [Windows Hello](#)
- [Credential locker](#)
- [Web authentication broker](#)
- [Fingerprint biometrics](#)
- [Smart cards](#)
- [Shared certificates](#)
- [Cryptography](#)
- [Certificates](#)
- [Cryptographic keys](#)
- [Data protection](#)
- [MACs, hashes, and signatures](#)
- [Export restrictions on cryptography](#)
- [Common cryptography tasks](#)

### 6.2 Code samples

- [Credential locker](#) 

- [Credential picker](#)
- [Device lockdown with Azure login](#)
- [Enterprise data protection](#)
- [KeyCredentialManager](#)
- [Smart cards](#)
- [Web account management](#)
- [WebAuthenticationBroker](#)

## 6.3 API reference

- [Windows.Security.Authentication.OnlineId](#)
- [Windows.Security.Authentication.Web](#)
- [Windows.Security.Authentication.Web.Core](#)
- [Windows.Security.Authentication.Web.Provider](#)
- [Windows.Security.Credentials](#)
- [Windows.Security.Credentials](#)
- [Windows.Security.Credentials.UI](#)
- [Windows.Security.Cryptography](#)
- [Windows.Security.Cryptography.Certificates](#)
- [Windows.Security.Cryptography.Core](#)
- [Windows.Security.Cryptography.DataProtection](#)
- [Windows.Security.ExchangeActiveSyncProvisioning](#)
- [Windows.Security.EnterpriseData](#)

---

## Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

# Authentication and user identity

Article • 07/11/2024

Universal Windows Platform (UWP) apps have several options for user authentication, ranging from simple single sign-on (SSO) using [Web authentication broker](#) to highly secure two-factor authentication.

For regular app connections to third-party identity provider services, use the [Web authentication broker](#). For added convenience, use [Credential Locker](#) to save and roam the user's login information.

Enterprises using Windows should strongly consider using [Windows Hello](#), which enables highly secure two-factor authentication. If using Windows Hello is not possible, [Smart cards](#) and [Fingerprint biometrics](#) can add an additional layer of security.

 Expand table

| Topic                                                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Credential locker</a>                    | This article describes how apps can use the Credential Locker to securely store and retrieve user credentials, and roam them between devices with the user's Microsoft account.                                                                                                                                                                                                                                                                                                                               |
| <a href="#">Fingerprint biometrics</a>               | This article explains how to add fingerprint biometrics to your app. Including a request for fingerprint authentication when the user must consent to a particular action increases the security of your app. For example, you could require fingerprint authentication before authorizing an in-app purchase, or access to restricted resources. Fingerprint authentication is managed using the <a href="#">UserConsentVerifier</a> class in the <a href="#">Windows.Security.Credentials.UI</a> namespace. |
| <a href="#">Windows Hello</a>                        | This article describes the Windows Hello technology, and discusses how developers can implement this technology to protect their apps and backend services. It highlights specific capabilities of these technologies that help mitigate threats from conventional credentials and provides guidance about designing and deploying these technologies as part of your packaged Windows apps.                                                                                                                  |
| <a href="#">Create a Windows Hello login app</a>     | Part 1 of a complete walkthrough on how to create a packaged Windows app that uses Windows Hello as an alternative to traditional username and password authentication systems.                                                                                                                                                                                                                                                                                                                               |
| <a href="#">Create a Windows Hello login service</a> | Part 2 of a complete walkthrough on how to use Windows Hello as an alternative to traditional username and password authentication systems in packaged Windows apps.                                                                                                                                                                                                                                                                                                                                          |

| Topic                                                     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Smart cards</a>                               | This topic explains how apps can use smart cards to connect users to secure network services, including how to access physical smart card readers, create virtual smart cards, communicate with smart cards, authenticate users, reset user PINs, and remove or disconnect smart cards.                                                                                                                                                                                                                                        |
| <a href="#">Share certificates between apps</a>           | UWP apps that require secure authentication beyond a user Id and password combination can use certificates for authentication. Certificate authentication provides a high level of trust when authenticating a user. In some cases, a group of services will want to authenticate a user for multiple apps. This article shows how you can authenticate multiple apps using the same certificate, and how you can provide convenient code for a user to import a certificate that was provided to access secured web services. |
| <a href="#">Windows Unlock with companion IoT devices</a> | A companion device is a device that can act in conjunction with Windows to enhance the user authentication experience. Using the Companion Device Framework, a companion device can provide a rich experience even when Windows Hello is not available (for example, if the Windows machine lacks a camera for face authentication or fingerprint reader device).                                                                                                                                                              |
| <a href="#">Web account manager</a>                       | This article describes how to show the AccountsSettingsPane and connect your Universal Windows Platform (UWP) app to external identity providers, like Microsoft or Facebook, using the new Windows 10/11 Web Account Manager APIs. You'll learn how to request a user's permission to use their Microsoft account, obtain an access token, and use it to perform basic operations (like get profile data or upload files to their OneDrive).                                                                                  |
| <a href="#">Web authentication broker</a>                 | This article explains how to connect your app to an online identity provider that uses authentication protocols like OpenID or OAuth. The <a href="#">AuthenticateAsync</a> method sends a request to the online identity provider and gets back an access token that describes the provider resources to which the app has access.                                                                                                                                                                                            |

## Feedback

Was this page helpful?



[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

# Credential locker for Windows apps

Article • 08/09/2024

This article describes how Windows apps can use the Credential Locker to securely store and retrieve user credentials, and roam them between devices with the user's Microsoft account.

The Windows Runtime (WinRT) APIs for Credential Locker access are part of the [Windows Software Development Kit \(SDK\)](#). These APIs were created for use in Universal Windows Platform (UWP) apps, but they can also be used in WinUI apps or in packaged desktop apps, including WPF and Windows Forms. For more information about using WinRT APIs in your Windows desktop app, see [Call Windows Runtime APIs in desktop apps](#).

## Overview of the sample scenario

For example, you have an app that connects to a service to access protected resources such as media files, or social networking. Your service requires login information for each user. You've built UI into your app that gets the username and password for the user, which is then used to log the user into the service. Using the Credential Locker API, you can store the username and password for your user and easily retrieve them and log the user in automatically the next time they open your app, regardless of what device they're on.

User credentials stored in the Credential Locker do *not* expire, are *not* affected by the [ApplicationData.RoamingStorageQuota](#), and will *not* be cleared out due to inactivity like traditional roaming data. However, you can only store up to 20 credentials per app in the Credential Locker.

Credential Locker works a little differently for domain accounts. If there are credentials stored with your Microsoft account, and you associate that account with a domain account (such as the account that you use at work), your credentials will roam to that domain account. However, any new credentials added when signed on with the domain account won't roam. This ensures that private credentials for the domain aren't exposed outside of the domain.

## Storing user credentials

1. Obtain a reference to the Credential Locker using the [PasswordVault](#) object from the [Windows.Security.Credentials](#) namespace.

2. Create a [PasswordCredential](#) object that contains an identifier for your app, the username and the password, and pass that to the [PasswordVault.Add](#) method to add the credential to the locker.

CS

```
var vault = new Windows.Security.Credentials.PasswordVault();
vault.Add(new Windows.Security.Credentials.PasswordCredential(
    "My App", username, password));
```

## Retrieving user credentials

You have several options for retrieving user credentials from the Credential Locker after you have a reference to the [PasswordVault](#) object.

- You can retrieve all the credentials the user has supplied for your app in the locker with the [PasswordVault.RetrieveAll](#) method.
- If you know the username for the stored credentials, you can retrieve all the credentials for that username with the [PasswordVault.FindAllByUserName](#) method.
- If you know the resource name for the stored credentials, you can retrieve all the credentials for that resource name with the [PasswordVault.FindAllByResource](#) method.
- Finally, if you know both the username and the resource name for a credential, you can retrieve just that credential with the [PasswordVault.Retrieve](#) method.

Let's look at an example where we have stored the resource name globally in an app and we log the user on automatically if we find a credential for them. If we find multiple credentials for the same user, we ask the user to select a default credential to use when logging on.

CS

```
private string resourceName = "My App";
private string defaultUserName;

private void Login()
{
    var loginCredential = GetCredentialFromLocker();

    if (loginCredential != null)
    {
        // There is a credential stored in the locker.
        // Populate the Password property of the credential
        // for automatic login.
        loginCredential.RetrievePassword();
    }
}
```

```

else
{
    // There is no credential stored in the locker.
    // Display UI to get user credentials.
    loginCredential = GetLoginCredentialUI();
}

// Log the user in.
ServerLogin(loginCredential.UserName, loginCredential.Password);
}

private Windows.Security.Credentials.PasswordCredential
GetCredentialFromLocker()
{
    Windows.Security.Credentials.PasswordCredential credential = null;

    var vault = new Windows.Security.Credentials.PasswordVault();

    IReadOnlyList<PasswordCredential> credentialList = null;

    try
    {
        credentialList = vault.FindAllByResource(resourceName);
    }
    catch(Exception)
    {
        return null;
    }

    if (credentialList.Count > 0)
    {
        if (credentialList.Count == 1)
        {
            credential = credentialList[0];
        }
        else
        {
            // When there are multiple usernames,
            // retrieve the default username. If one doesn't
            // exist, then display UI to have the user select
            // a default username.
            defaultUserName = GetDefaultUserNameUI();

            credential = vault.Retrieve(resourceName, defaultUserName);
        }
    }

    return credential;
}

```

## Deleting user credentials

Deleting user credentials in the Credential Locker is also a quick, two-step process.

1. Obtain a reference to the Credential Locker using the [PasswordVault](#) object from the [Windows.Security.Credentials](#) namespace.
2. Pass the credential you want to delete to the [PasswordVault.Remove](#) method.

CS

```
var vault = new Windows.Security.Credentials.PasswordVault();
vault.Remove(new Windows.Security.Credentials.PasswordCredential(
    "My App", username, password));
```

## Best practices

Only use the credential locker for passwords and not for larger data blobs.

Save passwords in the credential locker only if the following criteria are met:

- The user has successfully signed in.
- The user has opted to save passwords.

Never store credentials in plain-text using app data or roaming settings.

## Related content

- [PasswordVault](#)
- [Call Windows Runtime APIs in desktop apps](#)

### Collaborate with us on GitHub

The source for this content can be found on GitHub, where you can also create and review issues and pull requests. For more information, see [our contributor guide](#).



### Windows developer feedback

Windows developer is an open source project. Select a link to provide feedback:



[Open a documentation issue](#)



[Provide product feedback](#)

# Fingerprint biometrics

Article • 08/09/2024

This article explains how to add fingerprint biometrics to your Windows app, including a request for fingerprint authentication when the user must consent to a particular action increases the security of your app. For example, you could require fingerprint authentication before authorizing an in-app purchase, or access to restricted resources. Fingerprint authentication is managed using the [UserConsentVerifier](#) class in the [Windows.Security.Credentials.UI](#) namespace.

The Windows Runtime (WinRT) APIs for fingerprint biometrics are part of the [Windows Software Development Kit \(SDK\)](#). These APIs were created for use in Universal Windows Platform (UWP) apps, but they can also be used in WinUI apps or in packaged desktop apps, including WPF and Windows Forms. For more information about using WinRT APIs in your Windows desktop app, see [Call Windows Runtime APIs in desktop apps](#).

## Check the device for a fingerprint reader

To find out whether the device has a fingerprint reader, call [UserConsentVerifier.CheckAvailabilityAsync](#). Even if a device supports fingerprint authentication, your app should still provide users with an option in Settings to enable or disable it.

CS

```
public async System.Threading.Tasks.Task<string>
CheckFingerprintAvailability()
{
    string returnMessage = "";

    try
    {
        // Check the availability of fingerprint authentication.
        var ucvAvailability = await
Windows.Security.Credentials.UI.UserConsentVerifier.CheckAvailabilityAsync()
;

        switch (ucvAvailability)
        {
            case
Windows.Security.Credentials.UI.UserConsentVerifierAvailability.Available:
                returnMessage = "Fingerprint verification is available.";
                break;
            case
Windows.Security.Credentials.UI.UserConsentVerifierAvailability.DeviceBusy:
```

```

        returnMessage = "Biometric device is busy.";
        break;
    case
Windows.Security.Credentials.UI.UserConsentVerifierAvailability.DeviceNotPre
sent:
        returnMessage = "No biometric device found.";
        break;
    case
Windows.Security.Credentials.UI.UserConsentVerifierAvailability.DisabledByPo
licy:
        returnMessage = "Biometric verification is disabled by
policy.";
        break;
    case
Windows.Security.Credentials.UI.UserConsentVerifierAvailability.NotConfigure
dForUser:
        returnMessage = "The user has no fingerprints registered.
Please add a fingerprint to the " +
                        "fingerprint database and try again.";
        break;
    default:
        returnMessage = "Fingerprints verification is currently
unavailable.";
        break;
    }
}
catch (Exception ex)
{
    returnMessage = $"Fingerprint authentication availability check
failed: {ex.ToString()}";
}

return returnMessage;
}

```

## Request consent and return results

1. To request user consent from a fingerprint scan, call the [UserConsentVerifier.RequestVerificationAsync](#) method. For fingerprint authentication to work, the user must have previously added a fingerprint "signature" to the fingerprint database.
2. When you call the [UserConsentVerifier.RequestVerificationAsync](#), the user is presented with a modal dialog requesting a fingerprint scan. You can supply a message to the [UserConsentVerifier.RequestVerificationAsync](#) method that will be displayed to the user as part of the modal dialog, as shown in the following image.

```

private async System.Threading.Tasks.Task<string> RequestConsent(string
userMessage)
{
    string returnMessage;

    if (String.IsNullOrEmpty(userMessage))
    {
        userMessage = "Please provide fingerprint verification.";
    }

    try
    {
        // Request the logged on user's consent via fingerprint swipe.
        var consentResult = await
Windows.Security.Credentials.UI.UserConsentVerifier.RequestVerificationAsync
(userMessage);

        switch (consentResult)
        {
            case
Windows.Security.Credentials.UI.UserConsentVerificationResult.Verified:
                returnMessage = "Fingerprint verified.";
                break;
            case
Windows.Security.Credentials.UI.UserConsentVerificationResult.DeviceBusy:
                returnMessage = "Biometric device is busy.";
                break;
            case
Windows.Security.Credentials.UI.UserConsentVerificationResult.DeviceNotPrese
nt:
                returnMessage = "No biometric device found.";
                break;
            case
Windows.Security.Credentials.UI.UserConsentVerificationResult.DisabledByPoli
cy:
                returnMessage = "Biometric verification is disabled by
policy.";
                break;
            case
Windows.Security.Credentials.UI.UserConsentVerificationResult.NotConfiguredF
orUser:
                returnMessage = "The user has no fingerprints registered.
Please add a fingerprint to the " +
                "fingerprint database and try again.";
                break;
            case
Windows.Security.Credentials.UI.UserConsentVerificationResult.RetriesExhaust
ed:
                returnMessage = "There have been too many failed attempts.
Fingerprint authentication canceled.";
                break;
            case
Windows.Security.Credentials.UI.UserConsentVerificationResult.Canceled:
                returnMessage = "Fingerprint authentication canceled.";

```

```

        break;
    default:
        returnMessage = "Fingerprint authentication is currently
unavailable.";
        break;
    }
}
catch (Exception ex)
{
    returnMessage = $"Fingerprint authentication failed:
{ex.ToString()}";
}

return returnMessage;
}

```

## Related content

- [UserConsentVerifier](#)
- [Windows.Security.Credentials.UI namespace](#)
- [Call Windows Runtime APIs in desktop apps](#)

### Collaborate with us on GitHub

The source for this content can be found on GitHub, where you can also create and review issues and pull requests. For more information, see [our contributor guide](#).



### Windows developer feedback

Windows developer is an open source project. Select a link to provide feedback:

 [Open a documentation issue](#)

 [Provide product feedback](#)

# Windows Hello

Article • 07/11/2024

This article describes the Windows Hello technology that ships as part of Windows and discusses how developers can implement this technology to protect their Windows apps and backend services. It highlights specific capabilities of these technologies that help mitigate threats that arise from using conventional credentials and provides guidance about designing and deploying these technologies as part of a Windows client rollout.

## ⓘ Note

This article focuses on app development. For information on the architecture and implementation details of Windows Hello, see [Plan a Windows Hello for Business deployment](#).

For a step-by-step walkthrough on creating a WinUI app using Windows Hello and the backing authentication service, see the [Windows Hello login app](#) and [Windows Hello login service](#) articles.

## Introduction

A fundamental assumption about information security is that a system can identify who is using it. Identifying a user allows the system to decide whether the user is identified appropriately (a process known as authentication), and then decide what a properly authenticated user should be able to do (authorization). The overwhelming majority of computer systems deployed throughout the world depend on user credentials for making authentication and authorization decisions, which means that these systems depend on reusable, user-created passwords as the basis for their security. The oft-cited maxim that authentication can involve "something you know, something you have, or something you are" neatly highlights the issue: a reusable password is an authentication factor all by itself, so anyone who knows the password can impersonate the user who owns it.

## Problems with traditional credentials

Ever since the mid-1960s, when Fernando Corbató and his team at the Massachusetts Institute of Technology championed the introduction of the password, users and administrators have had to deal with the use of passwords for user authentication and authorization. Over time, the state of the art for password storage and use has advanced

somewhat (with secure hashing and salting, for example), but we are still faced with two problems. Passwords are easy to clone and they are easy to steal. In addition, implementation faults may render them insecure, and users have a hard time balancing convenience and security.

## **Credential theft**

The biggest risk of passwords is simple: an attacker can steal them easily. Every place a password is entered, processed, or stored is vulnerable. For example, an attacker can steal a collection of passwords or hashes from an authentication server by eavesdropping on network traffic to an application server, by implanting malware in an application or on a device, by logging user keystrokes on a device, or by watching to see which characters a user types. These are just the most common attack methods.

Another related risk is that of credential replay, in which an attacker captures a valid credential by eavesdropping on an insecure network, and then replays it later to impersonate a valid user. Most authentication protocols (including Kerberos and OAuth) protect against replay attacks by including a time stamp in the credential exchange process, but that tactic only protects the token that the authentication system issues, not the password that the user provides to get the ticket in the first place.

## **Credential reuse**

The common approach of using an email address as the username makes a bad problem worse. An attacker who successfully recovers a username–password pair from a compromised system can then try that same pair on other systems. This tactic works surprisingly often to allow attackers to springboard from a compromised system into other systems. The use of email addresses as usernames leads to additional problems that we will explore later in this guide.

## **Solving credential problems**

Solving the problems that passwords pose is tricky. Tightening password policies alone will not do it; users may just recycle, share, or write down passwords. Although user education is critical for authentication security, education alone does not eliminate the problem either.

Windows Hello replaces passwords with strong two-factor authentication (2FA) by verifying existing credentials and by creating a device-specific credential that a biometric or PIN-based user gesture protects.

# What is Windows Hello?

Windows Hello is the name Microsoft has given to the new biometric sign-in system built into Windows. Because it is built directly into the operating system, Windows Hello allows face or fingerprint identification to unlock users' devices. Authentication happens when the user supplies his or her unique biometric identifier to access the device-specific credentials, which means that an attacker who steals the device can't log on to it unless that attacker has the PIN. The Windows secure credential store protects biometric data on the device. By using Windows Hello to unlock a device, the authorized user gains access to all of his or her Windows experience, apps, data, websites, and services.

The Windows Hello authenticator is known as a Hello. A Hello is unique to the combination of an individual device and a specific user. It does not roam across devices, is not shared with a server or calling app, and cannot easily be extracted from a device. If multiple users share a device, each user needs to set up his or her own account. Every account gets a unique Hello for that device. You can think of a Hello as a token you can use to unlock (or release) a stored credential. The Hello itself does not authenticate you to an app or service, but it releases credentials that can. In other words, the Hello is not a user credential but it is a second factor for the authenticating process.

## Windows Hello authentication

Windows Hello provides a robust way for a device to recognize an individual user, which addresses the first part of the path between a user and a requested service or data item. After the device has recognized the user, it still must authenticate the user before determining whether to grant access to a requested resource. Windows Hello provides strong 2FA that is fully integrated into Windows and replaces reusable passwords with the combination of a specific device, and a biometric gesture or PIN.

Windows Hello is not just a replacement for traditional 2FA systems, though. It is conceptually similar to smart cards: authentication is performed by using cryptographic primitives instead of string comparisons, and the user's key material is secure inside tamper-resistant hardware. Windows Hello does not require the extra infrastructure components required for smart card deployment, either. In particular, you do not need a Public Key Infrastructure (PKI) to manage certificates, if you do not currently have one. Windows Hello combines the major advantages of smart cards—deployment flexibility for virtual smart cards and robust security for physical smart cards—without any of their drawbacks.

## How Windows Hello works

When the user sets up Windows Hello on his or her machine, it generates a new public–private key pair on the device. The [trusted platform module](#) (TPM) generates and protects this private key. If the device does not have a TPM chip, the private key is encrypted and protected by software. In addition TPM-enabled devices generate a block of data that can be used to attest that a key is bound to TPM. This attestation information can be used in your solution to decide if the user is granted a different authorization level for example.

To enable Windows Hello on a device, the user must have either their Microsoft Entra ID account or Microsoft Account connected in Windows settings.

## How keys are protected

Any time key material is generated, it must be protected against attack. The most robust way to do this is through specialized hardware. There is a long history of using hardware security modules (HSMs) to generate, store, and process keys for security-critical applications. Smart cards are a special type of HSM, as are devices that are compliant with the Trusted Computing Group TPM standard. Wherever possible, the Windows Hello implementation takes advantage of onboard TPM hardware to generate, store, and process keys. However, Windows Hello and Windows Hello for Work do not require an onboard TPM.

Whenever feasible, Microsoft recommends the use of TPM hardware. The TPM protects against a variety of known and potential attacks, including PIN brute-force attacks. The TPM provides an additional layer of protection after an account lockout as well. When the TPM has locked the key material, the user must reset the PIN. Resetting the PIN means that all keys and certificates encrypted with the old key material will be removed.

## Authentication

When a user wants to access protected key material, the authentication process begins with the user entering a PIN or biometric gesture to unlock the device, a process sometimes called "releasing the key".

An application can never use the keys from another application, nor can someone ever use the keys from another user. These keys are used to sign requests that are sent to the identity provider or IDP, seeking access to specified resources. Applications can use specific APIs to request operations that require key material for particular actions. Access through these APIs does require explicit validation through a user gesture, and the key material is not exposed to the requesting application. Rather, the application asks for a specific action like signing a piece of data, and the Windows Hello layer handles the actual work and returns the results.

# Getting ready to implement Windows Hello

Now that we have a basic understanding of how Windows Hello works, let us take a look at how to implement them in our own applications.

There are different scenarios we can implement using Windows Hello. For example, just logging on to your app on a device. The other common scenario would be to authenticate against a service. Instead of using a logon name and password, you will be using Windows Hello. In the following sections, we will discuss implementing a couple of different scenarios, including how to authenticate against your services with Windows Hello, and how to convert from an existing username/password system to a Windows Hello system.

## Implementing Windows Hello

In this section, we begin with a greenfield scenario with no existing authentication system, and we explain how to implement Windows Hello.

The next section covers how to migrate from an existing username/password system. However, even if that section interests you more, you may want to look through this one to get a basic understanding of the process and the code required.

## Enrolling new users

We begin with a brand new service that will use Windows Hello, and a hypothetical new user who is ready to sign up on a new device.

The first step is to verify that the user is able to use Windows Hello. The app verifies user settings and machine capabilities to make sure it can create user ID keys. If the app determines the user has not yet enabled Windows Hello, it prompts the user to set this up before using the app.

To enable Windows Hello, the user just needs to set up a PIN in Windows settings, unless the user set it up during the Out of Box Experience (OOBE).

The following lines of code show a simple way to check if the user is set up for Windows Hello.

C#

```
var keyCredentialAvailable = await KeyCredentialManager.IsSupportedAsync();
if (!keyCredentialAvailable)
{
    // User didn't set up PIN yet
}
```

```
    return;  
}
```

The next step is to ask the user for information to sign up with your service. You may choose to ask the user for first name, last name, email address, and a unique username. You could use the email address as the unique identifier; it is up to you.

In this scenario, we use the email address as the unique identifier for the user. Once the user signs up, you should consider sending a validation email to ensure the address is valid. This gives you a mechanism to reset the account if necessary.

If the user has set up his or her PIN, the app creates the user's [KeyCredential](#). The app also gets the optional key attestation information to acquire cryptographic proof that the key is generated on the TPM. The generated public key, and optionally the attestation, is sent to the backend server to register the device being used. Every key pair generated on every device will be unique.

The code to create the [KeyCredential](#) looks like this:

```
C#
```

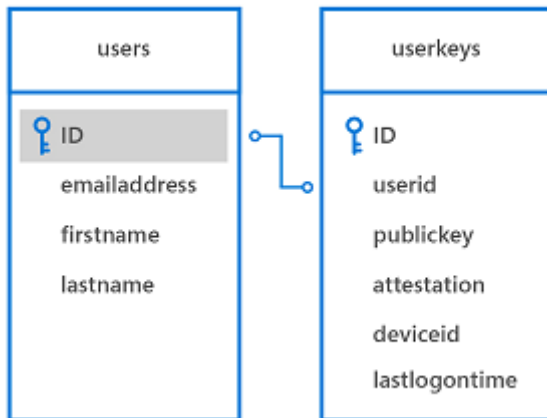
```
var keyCreationResult = await KeyCredentialManager.RequestCreateAsync(  
    AccountId, KeyCredentialCreationOption.ReplaceExisting);
```

The [RequestCreateAsync](#) is the call that creates the public and private key. If the device has the right TPM chip, the APIs will request the TPM chip to create the private and public key and store the result; if there is no TPM chip available, the OS will create the key pair in code. There is no way for the app to access the created private keys directly. Part of the creation of the key pairs is also the resulting Attestation information. (See the next section for more information about attestation.)

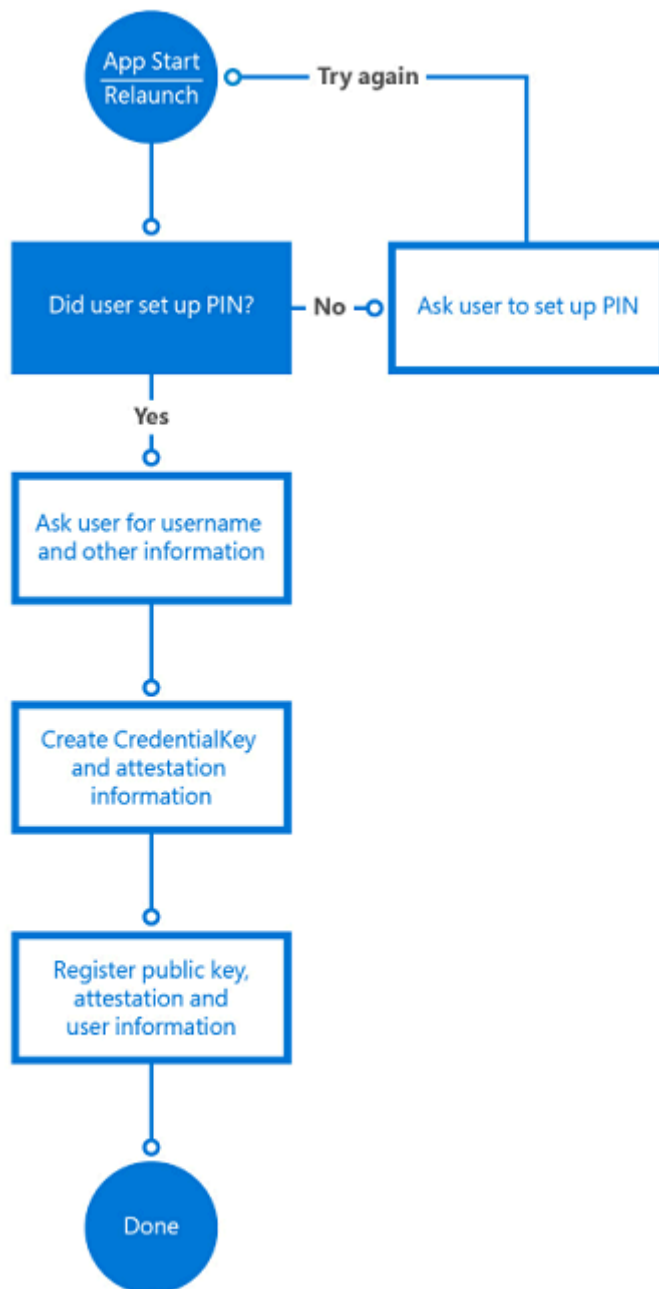
After the key pair and attestation information are created on the device, the public key, the optional attestation information, and the unique identifier (such as the email address) need to be sent to the backend registration service and stored in the backend.

To allow the user to access the app on multiple devices, the backend service needs to be able to store multiple keys for the same user. Because every key is unique for every device, we will store all these keys connected to the same user. A device identifier is used to help optimize the server part when authenticating users. We talk about this in more detail in the next section.

A sample database schema to store this information at the backend might look like this:



The registration logic might look like this:



The registration information you collect may of course include a lot more identifying information than we include in this simple scenario. For example, if your app accesses a secured service such as one for banking, you would need to request proof of identity and other things as part of the sign-up process. Once all the conditions are met, the public key of this user will be stored in the backend and used to validate the next time the user uses the service.

C#

```
using System;
using System.Runtime;
using System.Threading.Tasks;
using Windows.Storage.Streams;
using Windows.Security.Credentials;

static async void RegisterUser(string AccountId)
{
    var keyCredentialAvailable = await
    KeyCredentialManager.IsSupportedAsync();
    if (!keyCredentialAvailable)
    {
        // The user didn't set up a PIN yet
        return;
    }

    var keyCreationResult = await
    KeyCredentialManager.RequestCreateAsync(AccountId,
    KeyCredentialCreationOption.ReplaceExisting);
    if (keyCreationResult.Status == KeyCredentialStatus.Success)
    {
        var userKey = keyCreationResult.Credential;
        var publicKey = userKey.RetrievePublicKey();
        var keyAttestationResult = await userKey.GetAttestationAsync();
        IBuffer keyAttestation = null;
        IBuffer certificateChain = null;
        bool keyAttestationIncluded = false;
        bool keyAttestationCanBeRetrievedLater = false;

        keyAttestationResult = await userKey.GetAttestationAsync();
        KeyCredentialAttestationStatus keyAttestationRetryType = 0;

        switch (keyAttestationResult.Status)
        {
            case KeyCredentialAttestationStatus.Success:
                keyAttestationIncluded = true;
                keyAttestation = keyAttestationResult.AttestationBuffer;
                certificateChain =
                keyAttestationResult.CertificateChainBuffer;
                break;
            case KeyCredentialAttestationStatus.TemporaryFailure:
                keyAttestationRetryType =
                KeyCredentialAttestationStatus.TemporaryFailure;
        }
    }
}
```

```

        keyAttestationCanBeRetrievedLater = true;
        break;
    case KeyCredentialAttestationStatus.NotSupported:
        keyAttestationRetryType =
KeyCredentialAttestationStatus.NotSupported;
        keyAttestationCanBeRetrievedLater = true;
        break;
    }
}
else if (keyCreationResult.Status == KeyCredentialStatus.UserCanceled ||
        keyCreationResult.Status == KeyCredentialStatus.UserPrefersPassword)
{
    // Show error message to the user to get confirmation that user
    // does not want to enroll.
}
}

```

## Attestation

When creating the key pair, there is also an option to request the attestation information, which is generated by the TPM chip. This optional information can be sent to the server as part of the sign-up process. TPM key attestation is a protocol that cryptographically proves that a key is TPM-bound. This type of attestation can be used to guarantee that a certain cryptographic operation occurred in the TPM of a particular computer.

When it receives the generated RSA key, the attestation statement, and the AIK certificate, the server verifies the following conditions:

- The AIK certificate signature is valid.
- The AIK certificate chains up to a trusted root.
- The AIK certificate and its chain is enabled for EKU OID "2.23.133.8.3" (friendly name is "Attestation Identity Key Certificate").
- The AIK certificate is time valid.
- All issuing CA certificates in the chain are time-valid and not revoked.
- The attestation statement is formed correctly.
- The signature on [KeyAttestation](#) blob uses an AIK public key.
- The public key included in the [KeyAttestation](#) blob matches the public RSA key that client sent alongside the attestation statement.

Your app might assign the user a different authorization level, depending on these conditions. For instance, if one of these checks fail, it might not enroll the user or it might limit what the user can do.

## Logging on with Windows Hello

Once the user is enrolled in your system, he or she can use the app. Depending on the scenario, you can ask users to authenticate before they can start using the app or just ask them to authenticate once they start using your backend services.

## Force the user to sign in again

For some scenarios, you may want the user to prove he or she is the person who is currently signed in, before accessing the app or sometimes before performing a certain action inside of your app. For example, before a banking app sends the transfer money command to the server, you want to make sure it is the user, rather than someone who found a logged-in device, attempting to perform a transaction. You can force the user to sign in again in your app by using the [UserConsentVerifier](#) class. The following line of code will force the user to enter their credentials.

The following line of code will force the user to enter their credentials.

C#

```
UserConsentVerificationResult consentResult = await
UserConsentVerifier.RequestVerificationAsync("userMessage");
if (consentResult.Equals(UserConsentVerificationResult.Verified))
{
    // continue
}
```

You can also use the challenge response mechanism from the server, which requires a user to enter his or her PIN code or biometric credentials. It depends on the scenario you as a developer need to implement. This mechanism is described in the following section.

## Authentication at the backend

When the app attempts to access a protected backend service, the service sends a challenge to the app. The app uses the private key from the user to sign the challenge and sends it back to the server. Since the server has stored the public key for that user, it uses standard crypto APIs to make sure the message was indeed signed with the correct private key. On the client, the signing is done by the Windows Hello APIs; the developer will never have access to any user's private key.

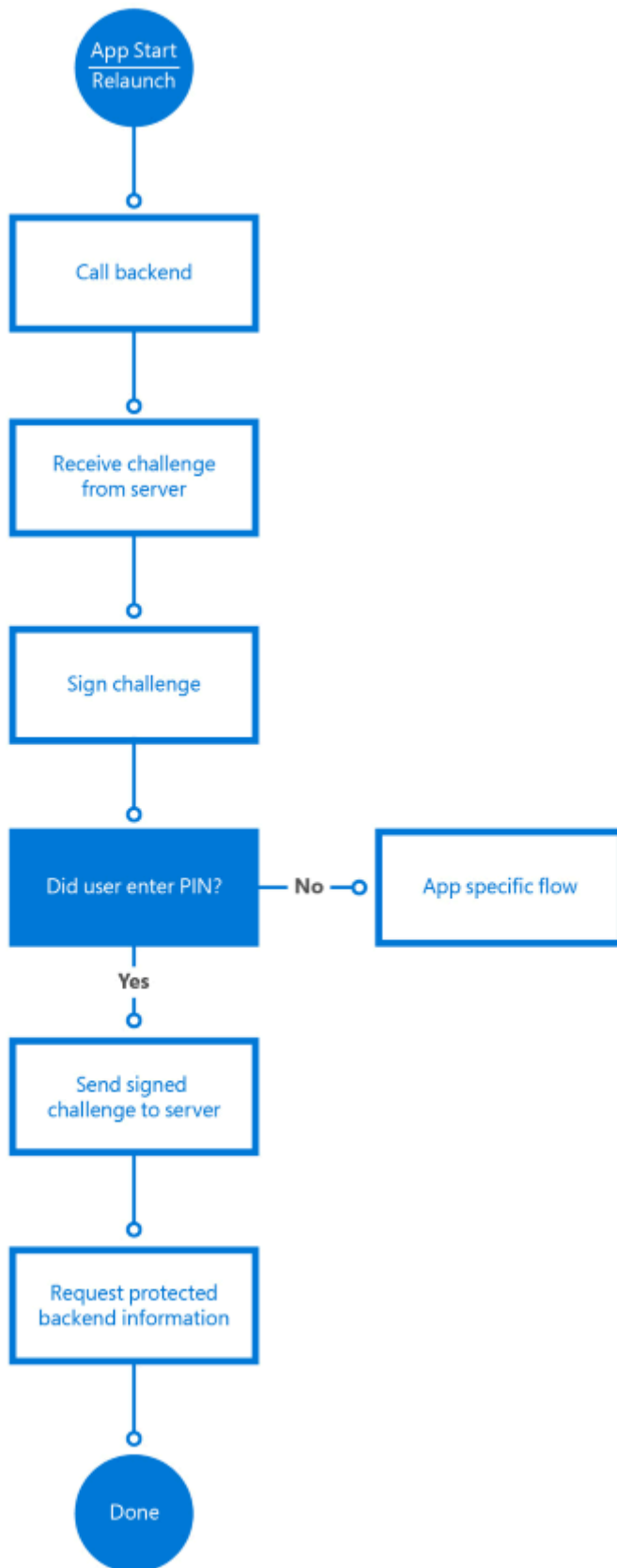
In addition to checking the keys, the service can also check the key attestation and discern if there are any limitations invoked on how the keys are stored on the device. For example, when the device uses TPM to protect the keys, it is more secure than devices storing the keys without TPM. The backend logic could decide, for example, that

the user is only allowed to transfer a certain amount of money when no TPM is used to reduce the risks.

Attestation is only available for devices with a TPM chip that's version 2.0 or later.

Therefore, you need to take into account that this information might not be available on every device.

The client workflow might look like the following chart:



When the app calls the service on the backend, the server sends a challenge. The challenge is signed with the following code:

C#

```
var openKeyResult = await KeyCredentialManager.OpenAsync(AccountId);  
  
if (openKeyResult.Status == KeyCredentialStatus.Success)
```

```

{
    var userKey = openKeyResult.Credential;
    var publicKey = userKey.RetrievePublicKey();
    var signResult = await userKey.RequestSignAsync(message);

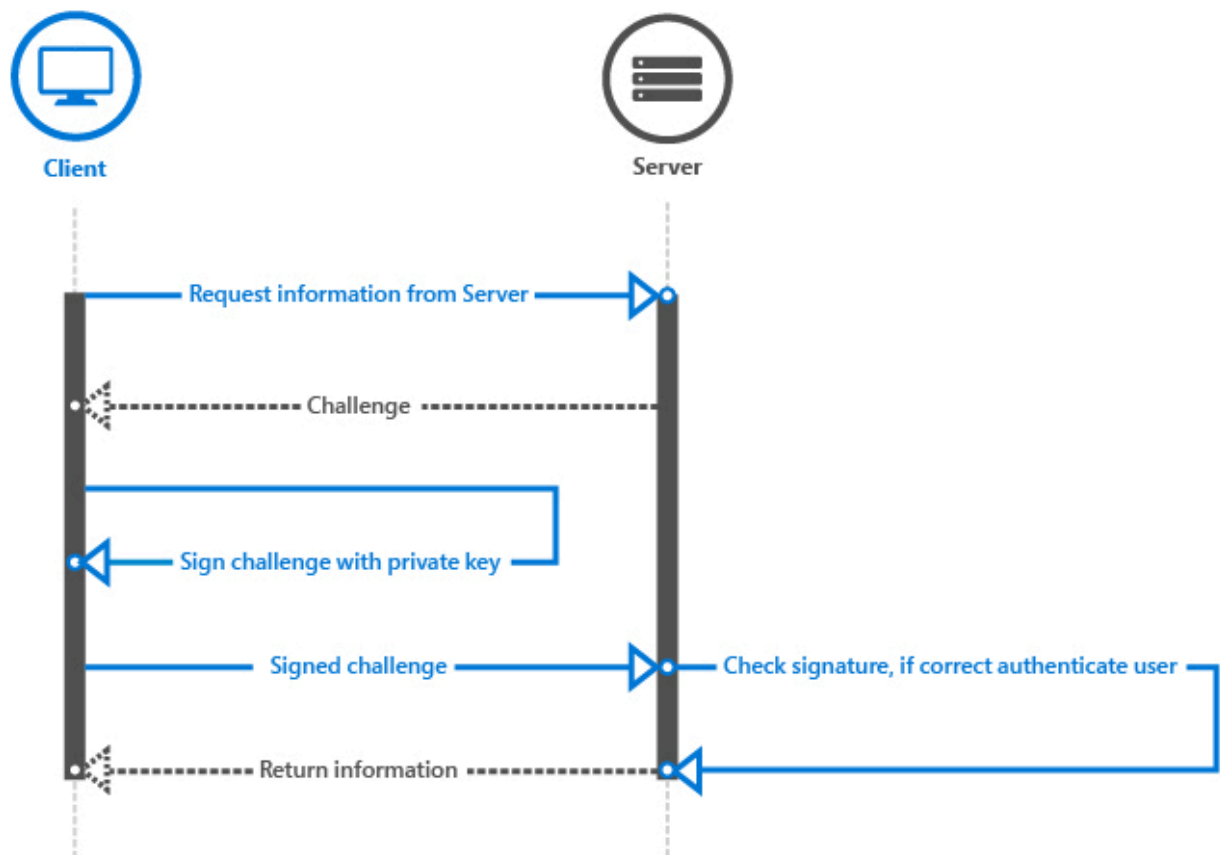
    if (signResult.Status == KeyCredentialStatus.Success)
    {
        return signResult.Result;
    }
    else if (signResult.Status == KeyCredentialStatus.UserPrefersPassword)
    {
    }
}

```

The first line, [KeyCredentialManager.OpenAsync](#), will ask Windows to open the key handle. If that's successful, you can sign the challenge message with the [KeyCredential.RequestSignAsync](#) method, causing Windows to request the user's PIN or biometrics through Windows Hello. At no time will the developer have access to the private key of the user. This is all kept secure through the APIs.

The APIs request Windows to sign the challenge with the private key. The system then asks the user for a PIN code or a configured biometric logon. When the correct information is entered, the system can ask the TPM chip to perform the cryptographic functions and sign the challenge. (Or use the fallback software solution, if no TPM is available). The client must send the signed challenge back to the server.

A basic challenge–response flow is shown in this sequence diagram:



Next, the server must validate the signature. When you request the public key and send it to the server to use for future validation, it is in an ASN.1-encoded publicKeyInfo blob. If you examine the [Windows Hello code sample on GitHub](#), you will see that there are helper classes to wrap Crypt32 functions to translate the ASN.1-encoded blob to a CNG blob, which is more commonly used. The blob contains the public key algorithm, which is RSA, and the RSA public key.

In the sample, the reason we convert the ASN.1-encoded blob to a CNG blob is so that it can be used with CNG and the BCrypt API. If you look up the CNG blob, it will point you to the related [BCRYPT\\_KEY\\_BLOB structure](#). This API surface can be used for authentication and encryption in Windows applications. ASN.1 is a documented standard for communicating data structures that can be serialized, and it's commonly used in public key cryptography and with certificates. That's why the public key information is returned in this manner. The public key is an RSA key; and that's the algorithm that Windows Hello uses when it signs data.

Once you have the CNG blob, you need to validate the signed challenge against the public key of the user. Since everyone uses his or her own system or backend technology, there is no generic way to implement this logic. We are using SHA256 as the hash algorithm and Pkcs1 for SignaturePadding, so make sure that's what you use when you validate the signed response from the client. Again, refer to the sample for a way to do it on your server in .NET 4.6, but in general it will look something like this:

C#

```
using (RSACng pubKey = new RSACng(publicKey))
{
    retval = pubKey.VerifyData(originalChallenge, responseSignature,
    HashAlgorithmName.SHA256, RSASignaturePadding.Pkcs1);
}
```

We read the stored public key, which is an RSA key. We validate the signed challenge message with the public key and if this checks out, we authorize the user. If the user is authenticated, the app can call the backend services as normal.

The complete code might look something like the following:

C#

```
using System;
using System.Runtime;
using System.Threading.Tasks;
using Windows.Storage.Streams;
using Windows.Security.Cryptography;
using Windows.Security.Cryptography.Core;
using Windows.Security.Credentials;

static async Task<IBuffer> GetAuthenticationMessageAsync(IBuffer message,
String AccountId)
{
    var openKeyResult = await KeyCredentialManager.OpenAsync(AccountId);

    if (openKeyResult.Status == KeyCredentialStatus.Success)
    {
        var userKey = openKeyResult.Credential;
        var publicKey = userKey.RetrievePublicKey();
        var signResult = await userKey.RequestSignAsync(message);
        if (signResult.Status == KeyCredentialStatus.Success)
        {
            return signResult.Result;
        }
        else if (signResult.Status == KeyCredentialStatus.UserCanceled)
        {
            // Launch app-specific flow to handle the scenario
            return null;
        }
    }
    else if (openKeyResult.Status == KeyCredentialStatus.NotFound)
    {
        // PIN reset has occurred somewhere else and key is lost.
        // Repeat key registration
        return null;
    }
    else
    {

```

```
        // Show custom UI because unknown error has happened.  
        return null;  
    }  
}
```

Implementing the correct challenge–response mechanism is outside the scope of this document, but this topic is something that requires attention in order to successfully create a secure mechanism to prevent things like replay attacks or man-in-the-middle attacks.

## Enrolling another device

It's common for users to have multiple devices with the same apps installed today. How does this work when using Windows Hello with multiple devices?

When using Windows Hello, every device will create a unique private and public key set. This means that if you want a user to be able to use multiple devices, your backend must be able to store multiple public keys from this user. Refer to the database diagram in the [Enrolling new users](#) section for an example of the table structure.

Registering another device is nearly the same as registering a user for the first time. You still need to ensure that the user registering for this new device is really the user they claim to be. You can do so with any two-factor authentication mechanism that is used today. There are several ways to accomplish this in a secure way. It all depends on your scenario.

For example, if you still use login name and password you can use that to authenticate the user and ask them to use one of their verification methods like SMS or email. If you don't have a login name and password, you can also use one of the already registered devices and send a notification to the app on that device. The MSA authenticator app is an example of this. In short, you should use a common 2FA mechanism to register extra devices for the user.

The code to register the new device is exactly the same as registering the user for the first time (from within the app).

C#

```
var keyCreationResult = await KeyCredentialManager.RequestCreateAsync(  
    AccountId, KeyCredentialCreationOption.ReplaceExisting);
```

To make it easier for the user to recognize which devices are registered, you can choose to send the device name or another identifier as part of the registration. This is also

useful, for example, if you want to implement a service on your backend where users can unregister devices when a device is lost.

## Using multiple accounts in your app

In addition to supporting multiple devices for a single account, it is also common to support multiple accounts in a single app. For example, maybe you are connecting to multiple Twitter accounts from within your app. With Windows Hello, you can create multiple key pairs and support multiple accounts inside your app.

One way of doing this is keeping the username or unique identifier described in the previous section on isolated storage. Therefore, every time you create a new account, you store the account ID in isolated storage.

In the app UI, you allow the user to either choose one of the previously created accounts or sign up with a new one. The flow of creating a new account is the same as described before. Choosing an account is a matter of listing the stored accounts on the screen. Once the user selects an account, use the account ID to log on the user in your app:

C#

```
var openKeyResult = await KeyCredentialManager.OpenAsync(AccountId);
```

The rest of the flow is the same as described earlier. To be clear, all these accounts are protected by the same PIN or biometric gesture since in this scenario they are being used on a single device with the same Windows Account.

## Migrating an Existing System to Windows Hello

In this short section, we will address an existing packaged app and backend system that uses a database that stores the username and hashed password. These apps collect credentials from the user when the app starts and use them when the backend system returns the authentication challenge.

Here, we will describe what pieces need to be changed or replaced to make Windows Hello work.

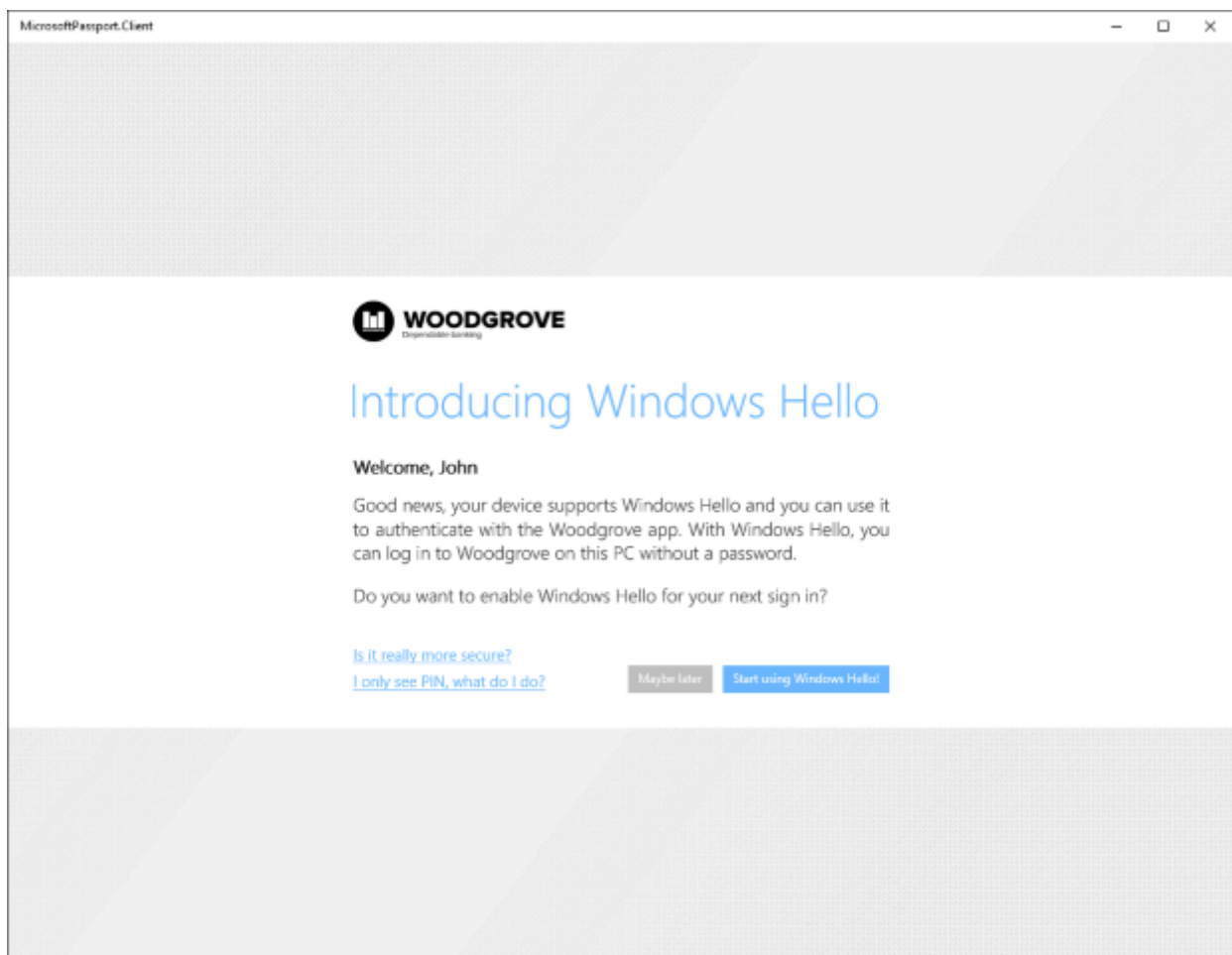
We have already described most of the techniques in the earlier sections. Adding Windows Hello to your existing system involves adding a couple of different flows in the registration and authentication part of your code.

One approach is to let the user choose when to upgrade. After the user logs on to the app and you detect that the app and OS are capable of supporting Windows Hello, you can ask the user if he or she wants to upgrade credentials to use this modern and more secure system. You can use the following code to check whether the user is capable of using Windows Hello.

C#

```
var keyCredentialAvailable = await KeyCredentialManager.IsSupportedAsync();
```

The UI might look something like this:



If the user elects to start using Windows Hello, you create the [KeyCredential](#) described before. The backend registration server adds the public key and optional attestation statement to the database. Because the user is already authenticated with username and password, the server can link the new credentials to the current user information in the database. The database model could be the same as the example described earlier.

If the app was able to create the users [KeyCredential](#), it stores the user ID in isolated storage so the user can pick this account from the list once the app is started again. From this point on, the flow exactly follows the examples described in earlier sections.

The final step in migrating to a full Windows Hello scenario is disabling the logon name and password option in the app and removing the stored hashed passwords from your database.

## Summary

Windows introduces a higher level of security that is also simple to put into practice. Windows Hello provides a new biometric sign-in system that recognizes the user and actively defeats efforts to circumvent proper identification. It can then deliver multiple layers of keys and certificates that can never be revealed or used outside the trusted platform module. In addition, a further layer of security is available through the optional use of attestation identity keys and certificates.

As a developer, you can use this guidance on design and deployment of these technologies to easily add secure authentication to your packaged Windows app rollouts to protect apps and backend services. The code required is minimal and easy to understand. Windows handles the heavy lifting.

Flexible implementation options allow Windows Hello to replace or work alongside your existing authentication system. The deployment experience is painless and economical. No additional infrastructure is needed to deploy Windows security. With Microsoft Hello built in to the operating system, Windows offers the most secure solution to the authentication problems facing the modern developer.

Mission accomplished! You just made the Internet a safer place!

## Additional resources

### Articles and sample code

- [Windows Hello overview](#) 
- [Plan a Windows Hello for Business deployment](#)
- [Windows Hello UWP code sample on GitHub](#) 

## Terminology

 Expand table

| Term                | Definition                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AIK                 | An attestation identity key is used to provide such a cryptographic proof (TPM key attestation) by signing the properties of the non-migratable key and providing the properties and signature to the relying party for verification. The resulting signature is called an “attestation statement.” Since the signature is created using the AIK private key—which can only be used in the TPM that created it—the relying party can trust that the attested key is truly non-migratable and cannot be used outside that TPM. |
| AIK Certificate     | An AIK certificate is used to attest to the presence of an AIK within a TPM. It is also used to attest that other keys certified by the AIK originated from that particular TPM.                                                                                                                                                                                                                                                                                                                                              |
| IDP                 | An IDP is an identity provider. An example is the IDP build by Microsoft for Microsoft Accounts. Every time an application needs to authenticate with an MSA, it can call the MSA IDP.                                                                                                                                                                                                                                                                                                                                        |
| PKI                 | Public key infrastructure is commonly used to point to an environment hosted by an organization itself and being responsible for creating keys, revoking keys, etc.                                                                                                                                                                                                                                                                                                                                                           |
| TPM                 | The trusted platform module can be used to create cryptographic public/private key pairs in such a way that the private key can never be revealed or used outside the TPM (that is, the key is non-migratable).                                                                                                                                                                                                                                                                                                               |
| TPM Key Attestation | A protocol that cryptographically proves that a key is TPM-bound. This type of attestation can be used to guarantee that a certain cryptographic operation occurred in the TPM of a particular computer                                                                                                                                                                                                                                                                                                                       |

## Related topics

- [Windows Hello login app](#)
- [Windows Hello login service](#)

### Collaborate with us on GitHub

The source for this content can be found on GitHub, where you can also create and review issues and pull requests. For more information, see [our contributor guide](#).



### Windows developer feedback

Windows developer is an open source project. Select a link to provide feedback:

 [Open a documentation issue](#)

 [Provide product feedback](#)

# Create a Windows Hello login app

Article • 07/11/2024

This is the first part of a complete walkthrough on how to create a packaged Windows app that uses Windows Hello as an alternative to traditional username and password authentication systems. In this case, the app is a WinUI app, but the same approach can be used with any packaged Windows app, including WPF and Windows Forms apps. The app uses a username for sign-in and creates a Hello key for each account. These accounts will be protected by the PIN that is set up in Windows Settings on configuration of Windows Hello.

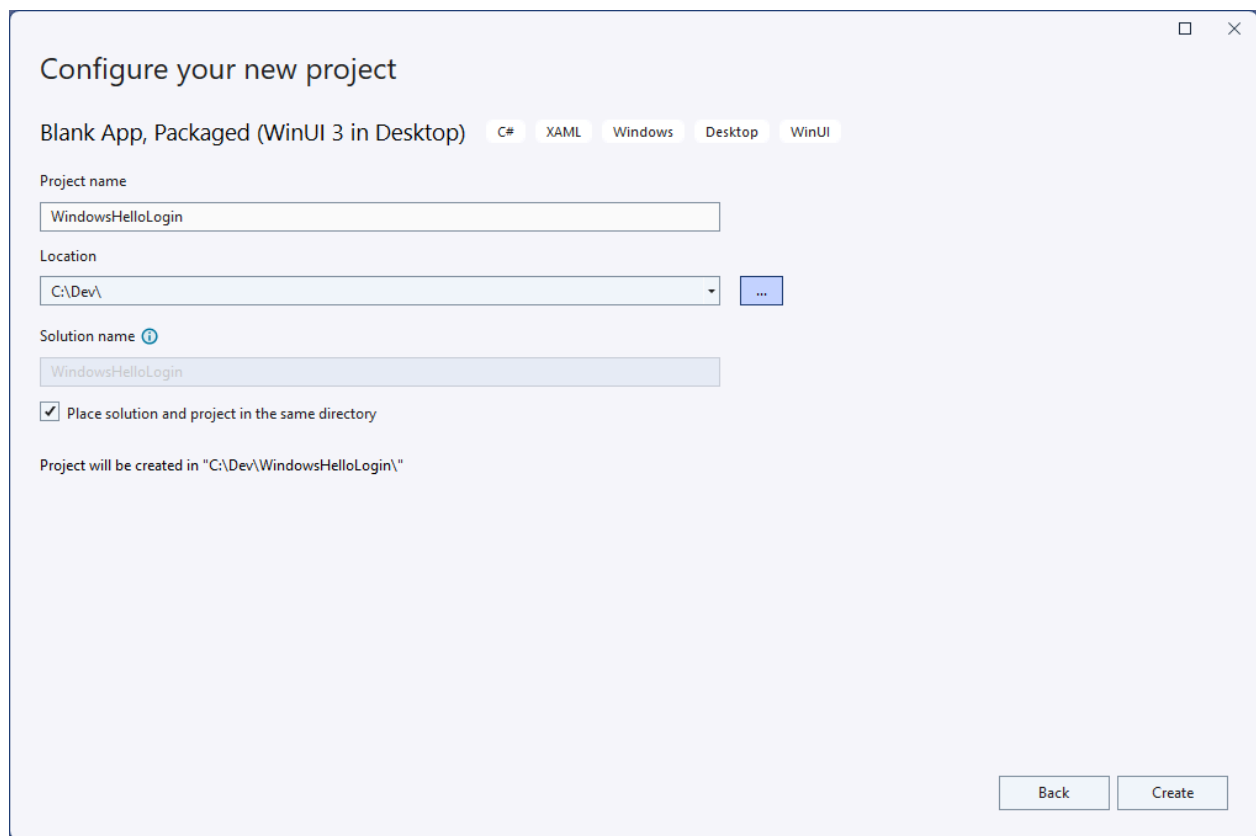
This walkthrough is split into two parts: building the app and connecting the backend service. When you're finished with this article, continue on to Part 2: [Windows Hello login service](#).

Before you begin, you should read the [Windows Hello](#) overview for a general understanding of how Windows Hello works.

## Get started

In order to build this project, you'll need some experience with C#, and XAML. You'll also need to be using Visual Studio 2022 on a Windows 10 or Windows 11 machine. See [Start developing Windows apps](#) for complete instructions on setting up your development environment.

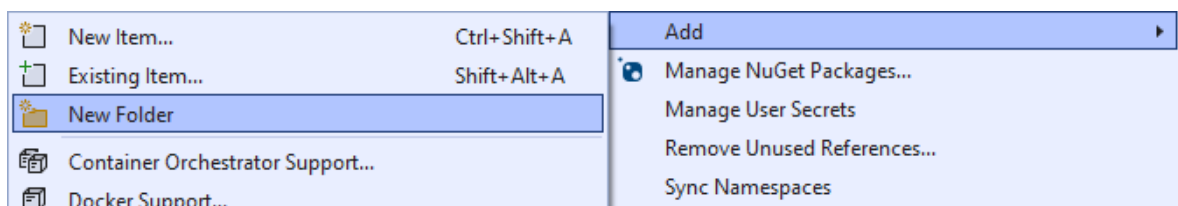
- In Visual Studio, select **File > New > Project**.
- In the **New Project** dialog's drop-down filters, select **C#/C++**, **Windows**, and **WinUI**, respectively.
- Choose **Blank App, Packaged (WinUI 3 in Desktop)** and name your application "WindowsHelloLogin".
- Build and Run the new application (F5), you should see a blank window shown on the screen. Close the application.



## Exercise 1: Login with Windows Hello

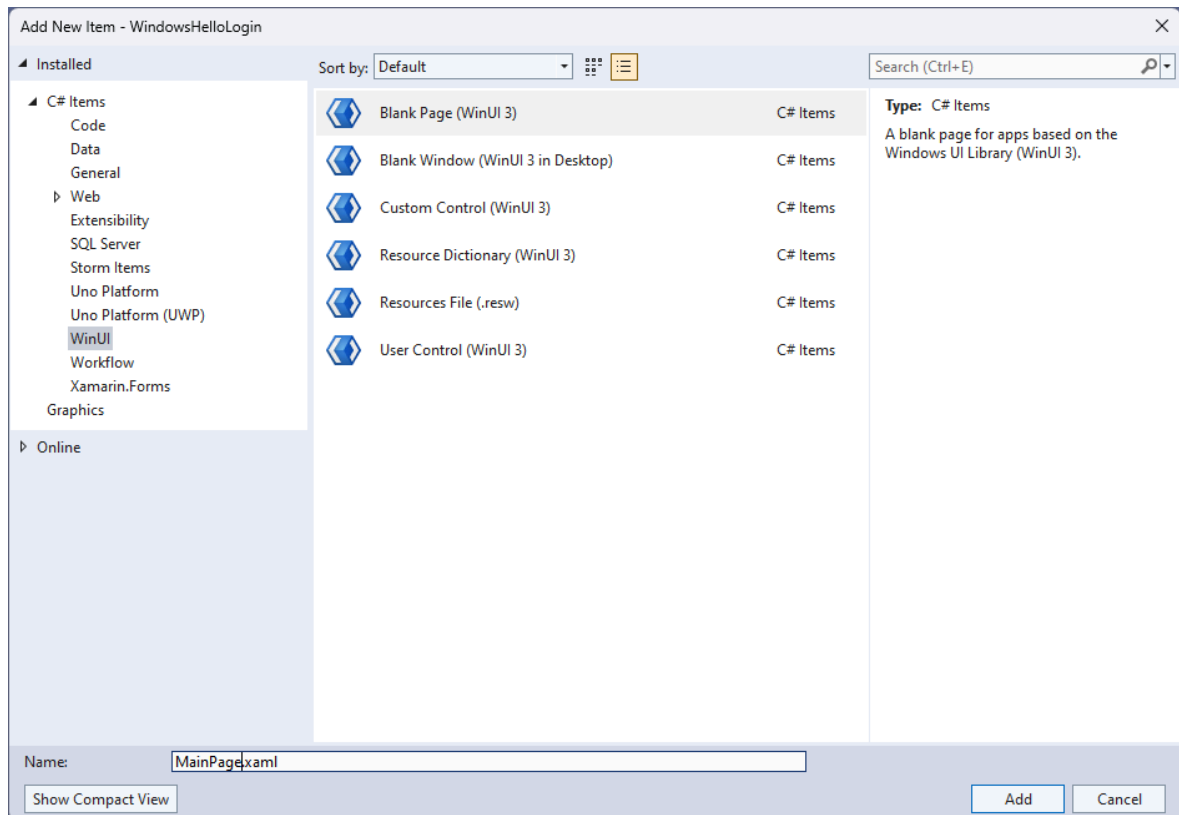
In this exercise you will learn how to check if Windows Hello is setup on the machine, and how to sign into an account using Windows Hello.

- In the new project create a new folder in the solution called "Views". This folder will contain the pages that will be navigated to in this sample. Right-click the project in **Solution Explorer**, select **Add > New Folder**, then rename the folder to **Views**.



- Open `MainWindow.xaml` and replace the `Window` contents with an empty `StackPanel` or `Grid` control. We will be implementing page navigation and navigating to a new page when the **MainWindow** is loaded, so we don't need any content in the **MainWindow**.
- Add a `Title` property to **MainWindow** in the XAML. The attribute should look like this: `Title="Windows Hello Login"`.
- Remove the `myButton_Click` event handler from `MainWindow.xaml.cs` to avoid any compilation errors. This event handler is not needed for this sample.

- Right-click the new **Views** folder, select **Add > New Item** and select the **Blank Page** template. Name this page "MainPage.xaml".



- Open the App.xaml.cs file and update the **OnLaunched** handler to implement page navigation for the app. You'll also need to add a **RootFrame\_NavigationFailed** handler method to deal with any errors that occur while loading pages.

```

CS

protected override void
OnLaunched(Microsoft.UI.Xaml.LaunchActivatedEventArgs args)
{
    m_window = new MainWindow();
    var rootFrame = new Frame();
    rootFrame.NavigationFailed += RootFrame_NavigationFailed;
    rootFrame.Navigate(typeof(MainPage), args);
    m_window.Content = rootFrame;
    m_window.Activate();
}

private void RootFrame_NavigationFailed(object sender,
NavigationFailedEventArgs e)
{
    throw new Exception($"Error loading page
{e.SourcePageType.FullName}");
}

```

- You'll also need to add four using statements to the top of the App.xaml.cs file to resolve compilation errors in the code.

CS

```
using Microsoft.UI.Xaml.Controls;  
using Microsoft.UI.Xaml.Navigation;  
using System;  
using WindowsHelloLogin.Views;
```

- Right-click the new **Views** folder, select **Add > New Item** and select the **Blank Page** template. Name this page "Login.xaml".
- To define the user interface for the new login page, add the following XAML. This XAML defines a `StackPanel` to align the following children:
  - A `TextBlock` that will contain a title.
  - A `TextBlock` for error messages.
  - A `TextBox` for the username to be input.
  - A `Button` to navigate to a register page.
  - A `TextBlock` to contain the status of Windows Hello.
  - A `TextBlock` to explain the Login page, as there is no backend or configured users yet.

XML

```
<Grid>  
    <StackPanel>  
        <TextBlock Text="Login" FontSize="36" Margin="4"  
TextAlignment="Center"/>  
        <TextBlock x:Name="ErrorMessage" Text="" FontSize="20"  
Margin="4" Foreground="Red" TextAlignment="Center"/>  
        <TextBlock Text="Enter your username below" Margin="0,0,0,20"  
TextWrapping="Wrap" Width="300"  
TextAlignment="Center" VerticalAlignment="Center"  
FontSize="16"/>  
        <TextBox x:Name="UsernameTextBox" Margin="4" Width="250"/>  
        <Button x:Name="LoginButton" Content="Login"  
Background="DodgerBlue" Foreground="White"  
Click="LoginButton_Click" Width="80"  
HorizontalAlignment="Center" Margin="0,20"/>  
        <TextBlock Text="Don't have an account?"  
TextAlignment="Center" VerticalAlignment="Center"  
FontSize="16"/>  
        <TextBlock x:Name="RegisterButtonTextBlock" Text="Register now"  
PointerPressed="RegisterButtonTextBlock_OnPointerPressed"  
Foreground="DodgerBlue"
```

```

        TextAlignment="Center" VerticalAlignment="Center"
        FontSize="16"/>
        <Border x:Name="WindowsHelloStatus" Background="#22B14C"
            Margin="0,20" Height="100" >
            <TextBlock x:Name="WindowsHelloStatusText" Text="Windows Hello
            is ready to use!"
                Margin="4" TextAlignment="Center"
                VerticalAlignment="Center" FontSize="20"/>
            </Border>
            <TextBlock x:Name="LoginExplanation" FontSize="24"
                TextAlignment="Center" TextWrapping="Wrap"
                Text="Please Note: To demonstrate a login, validation
                will only occur using the default username 'sampleUsername'"/>
        </StackPanel>
    </Grid>

```

- A few methods need to be added to the code-behind file to get the solution building. Either press F7 or use the **Solution Explorer** to edit the Login.xaml.cs file. Add in the following two event methods to handle the **Login** and **Register** events. For now these methods will set the `ErrorMessage.Text` to an empty string. Be sure to include the following using statements. They will be needed for the next steps.

CS

```

using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;
using Microsoft.UI.Xaml.Input;
using Microsoft.UI.Xaml.Media;
using Microsoft.UI.Xaml.Navigation;
using System;

namespace WindowsHelloLogin.Views
{
    public sealed partial class Login : Page
    {
        public Login()
        {
            this.InitializeComponent();
        }
        private void LoginButton_Click(object sender, RoutedEventArgs
e)
        {
            ErrorMessage.Text = "";
        }
        private void RegisterButtonTextBlock_OnPointerPressed(object
sender, PointerRoutedEventArgs e)
        {
            ErrorMessage.Text = "";
        }
    }
}

```

- In order to render the **Login** page, edit the **MainPage** code to navigate to the **Login** page when the **MainPage** is loaded. Open the MainPage.xaml.cs file. In **Solution Explorer**, double-click on MainPage.xaml.cs. If you can't find this click the little arrow next to MainPage.xaml to show the code-behind file. Create a **Loaded** event handler method that will navigate to the **Login** page.

CS

```
namespace WindowsHelloLogin.Views
{
    public sealed partial class MainPage : Page
    {
        public MainPage()
        {
            this.InitializeComponent();
            Loaded += MainPage_Loaded;
        }

        private void MainPage_Loaded(object sender, RoutedEventArgs e)
        {
            Frame.Navigate(typeof(Login));
        }
    }
}
```

- In the **Login** page, you need to handle the **OnNavigatedTo** event to validate if Windows Hello is available on the current machine. In Login.xaml.cs, implement the following code. You'll notice that the WindowsHelloHelper object indicates that there's an error. That's because we haven't created this helper class yet.

CS

```
public sealed partial class Login : Page
{
    public Login()
    {
        this.InitializeComponent();
    }

    protected override async void OnNavigatedTo(NavigationEventArgs e)
    {
        // Check if Windows Hello is set up and available on this
        machine
        if (await WindowsHelloHelper.WindowsHelloAvailableCheckAsync())
        {
        }
        else
        {
            // Windows Hello isn't set up, so inform the user
            WindowsHelloStatus.Background = new

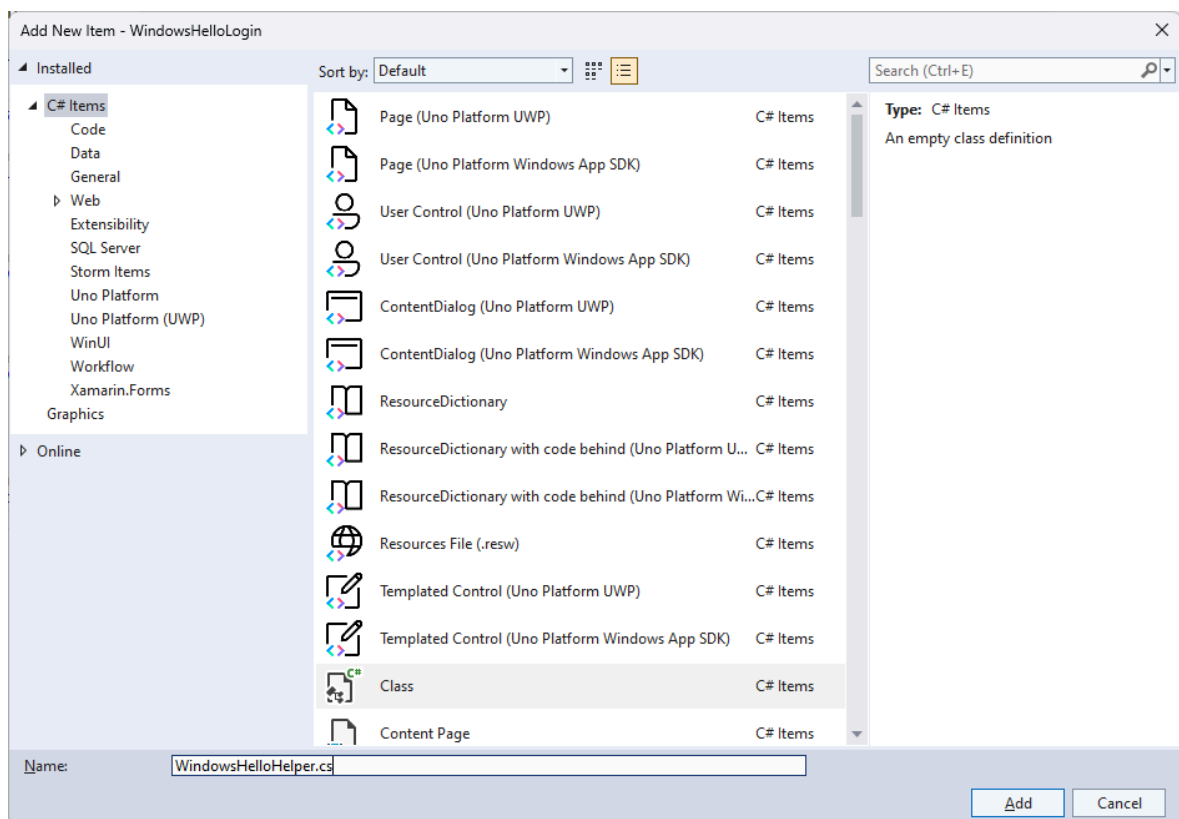
```

```

SolidColorBrush(Windows.UI.Color.FromArgb(255, 50, 170, 207));
        WindowsHelloStatusText.Text = $"Windows Hello is not set
up!{Environment.NewLine}Please go to Windows Settings and set up a PIN
to use it.";
        LoginButton.IsEnabled = false;
    }
}
}

```

- To create the WindowsHelloHelper class, right-click the **WindowsHelloLogin** project and click **Add > New Folder**. Name this folder **Utils**.
- Right click the **Utils** folder and select **Add > Class**. Name this new class "WindowsHelloHelper.cs".



- Change the scope of the **WindowsHelloHelper** class to be `public static`, then add the following method that to inform the user if Windows Hello is ready to be used or not. You will need to add the required namespaces.

CS

```

using System;
using System.Diagnostics;
using System.Threading.Tasks;
using Windows.Security.Credentials;

namespace WindowsHelloLogin.Utils
{
    public static class WindowsHelloHelper

```

```

{
    /// <summary>
    /// Checks to see if Windows Hello is ready to be used.
    ///
    /// Windows Hello has dependencies on:
    ///     1. Having a connected Microsoft Account
    ///     2. Having a Windows PIN set up for that account on the
    local machine
    /// </summary>
    public static async Task<bool>
    WindowsHelloAvailableCheckAsync()
    {
        bool keyCredentialAvailable = await
        KeyCredentialManager.IsSupportedAsync();
        if (keyCredentialAvailable == false)
        {
            // Key credential is not enabled yet as user
            // needs to connect to a Microsoft Account and select a
            PIN in the connecting flow.
            Debug.WriteLine("Windows Hello is not set up!\nPlease
            go to Windows Settings and set up a PIN to use it.");
            return false;
        }

        return true;
    }
}

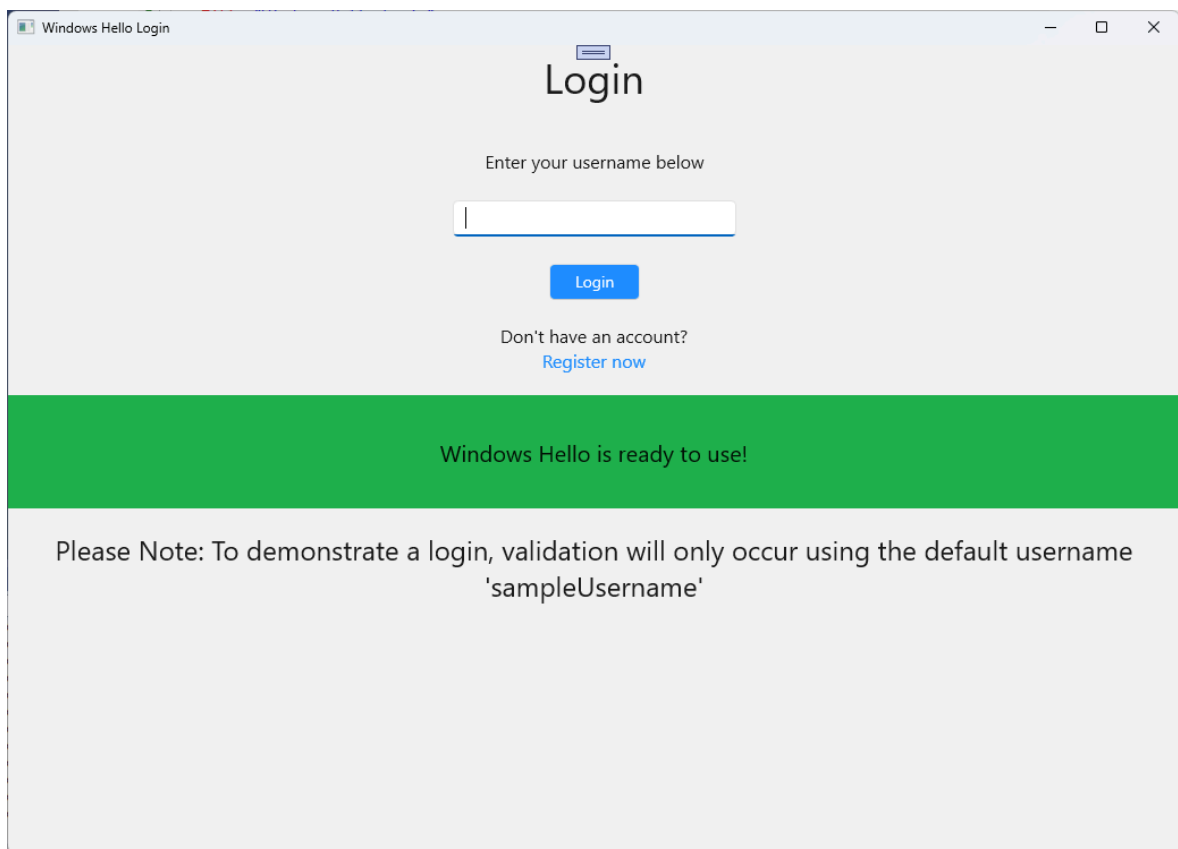
```

- In Login.xaml.cs, add a reference to the `WindowsHelloLogin.Utills` namespace. This will resolve the error in the `OnNavigatedTo` method.

CS

```
using WindowsHelloLogin.Utills;
```

- Build and run the application. You will be navigated to the login page and the Windows Hello banner will indicate to you if Windows Hello is ready to be used. You should see either the green or blue banner indicating the Windows Hello status on your machine.



- The next thing you need to do is build the logic for signing in. Create a new folder in the project named "Models".
- In the **Models** folder, create a new class called "Account.cs". This class will act as your account model. As this is a sample project, it will only contain a username. Change the class scope to `public` and add the `Username` property.

```
CS

namespace WindowsHelloLogin.Models
{
    public class Account
    {
        public string Username { get; set; }
    }
}
```

- The app needs a way to handle accounts. For this hands on lab, as there is no server or database, a list of users are saved and loaded locally. Right-click the **Utils** folder and add a new class named "AccountHelper.cs". Change the class scope to be `public static`. The **AccountHelper** is a static class that contains all the necessary methods to save and load the list of accounts locally. Saving and loading works by using an **XmlSerializer**. You also need to remember the file that was saved and where you saved it.

```
CS
```

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using System.Threading.Tasks;
using System.Xml.Serialization;
using Windows.Storage;
using WindowsHelloLogin.Models;

namespace WindowsHelloLogin.Utils
{
    public static class AccountHelper
    {
        // In the real world this would not be needed as there would be
        // a server implemented that would host a user account database.
        // For this tutorial we will just be storing accounts locally.
        private const string USER_ACCOUNT_LIST_FILE_NAME =
"accountlist.txt";
        private static string _accountListPath =
Path.Combine(ApplicationData.Current.LocalFolder.Path,
USER_ACCOUNT_LIST_FILE_NAME);
        public static List<Account> AccountList = [];

        /// <summary>
        /// Create and save a useraccount list file. (Updating the old
one)
        /// </summary>
        private static async void SaveAccountListAsync()
        {
            string accountsXml = SerializeAccountListToXml();

            if (File.Exists(_accountListPath))
            {
                StorageFile accountsFile = await
StorageFile.GetFileFromPathAsync(_accountListPath);
                await FileIO.WriteTextAsync(accountsFile, accountsXml);
            }
            else
            {
                StorageFile accountsFile = await
ApplicationData.Current.LocalFolder.CreateFileAsync(USER_ACCOUNT_LIST_F
ILE_NAME);
                await FileIO.WriteTextAsync(accountsFile, accountsXml);
            }
        }

        /// <summary>
        /// Gets the useraccount list file and deserializes it from XML
to a list of useraccount objects.
        /// </summary>
        /// <returns>List of useraccount objects</returns>
        public static async Task<List<Account>> LoadAccountListAsync()
        {
            if (File.Exists(_accountListPath))

```

```

        {
            StorageFile accountsFile = await
StorageFile.GetFileFromPathAsync(_accountListPath);

            string accountsXml = await
FileIO.ReadTextAsync(accountsFile);
            DeserializeXmlToAccountList(accountsXml);
        }

        return AccountList;
    }

    /// <summary>
    /// Uses the local list of accounts and returns an XML
formatted string representing the list
    /// </summary>
    /// <returns>XML formatted list of accounts</returns>
    public static string SerializeAccountListToXml()
    {
        var xmlizer = new XmlSerializer(typeof(List<Account>));
        var writer = new StringWriter();
        xmlizer.Serialize(writer, AccountList);

        return writer.ToString();
    }

    /// <summary>
    /// Takes an XML formatted string representing a list of
accounts and returns a list object of accounts
    /// </summary>
    /// <param name="listAsXml">XML formatted list of
accounts</param>
    /// <returns>List object of accounts</returns>
    public static List<Account> DeserializeXmlToAccountList(string
listAsXml)
    {
        var xmlizer = new XmlSerializer(typeof(List<Account>));
        TextReader textreader = new StreamReader(new
MemoryStream(Encoding.UTF8.GetBytes(listAsXml)));

        return AccountList = (xmlizer.Deserialize(textreader)) as
List<Account>;
    }
}
}

```

- Next, implement a way to add and remove an account from the local list of accounts. These actions will each save the list. The final method that you'll need for this hands-on lab is a validation method. As there is no authorization server or database of users, this will validate against a single user which is hard-coded. These methods should be added to the **AccountHelper** class.

```

public static Account AddAccount(string username)
{
    // Create a new account with the username
    var account = new Account() { Username = username };
    // Add it to the local list of accounts
    AccountList.Add(account);
    // SaveAccountList and return the account
    SaveAccountListAsync();
    return account;
}

public static void RemoveAccount(Account account)
{
    // Remove the account from the accounts list
    AccountList.Remove(account);
    // Re save the updated list
    SaveAccountListAsync();
}

public static bool ValidateAccountCredentials(string username)
{
    // In the real world, this method would call the server to
    // authenticate that the account exists and is valid.
    // However, for this tutorial, we'll just have an existing sample
    // user that's named "sampleUsername".
    // If the username is null or does not match "sampleUsername"
    // validation will fail.
    // In this case, the user should register a new Windows Hello user.

    if (string.IsNullOrEmpty(username))
    {
        return false;
    }

    if (!string.Equals(username, "sampleUsername"))
    {
        return false;
    }

    return true;
}

```

- The next thing you need to do is handle a sign in request from the user. In Login.xaml.cs, create a new private variable that will hold the current account logging in. Then add a new method named **SignInWindowsHelloAsync**. This will validate the account credentials using the **AccountHelper.ValidateAccountCredentials** method. This method will return a **Boolean** value if the entered user name is the same as the hard coded string value

you configured in the previous step. The hard-coded value for this sample is "sampleUsername".

CS

```
using WindowsHelloLogin.Models;
using WindowsHelloLogin.Utls;
using System.Diagnostics;
using System.Threading.Tasks;

namespace WindowsHelloLogin.Views
{
    public sealed partial class Login : Page
    {
        private Account _account;

        public Login()
        {
            this.InitializeComponent();
        }

        protected override async void OnNavigatedTo(NavigationEventArgs e)
        {
            // Check if Windows Hello is set up and available on this machine
            if (await WindowsHelloHelper.WindowsHelloAvailableCheckAsync())
            {
            }
            else
            {
                // Windows Hello is not set up, so inform the user
                WindowsHelloStatus.Background = new SolidColorBrush(Windows.UI.Color.FromArgb(255, 50, 170, 207));
                WindowsHelloStatusText.Text = "Windows Hello is not set up!\nPlease go to Windows Settings and set up a PIN to use it.";
                LoginButton.IsEnabled = false;
            }
        }

        private async void LoginButton_Click(object sender, RoutedEventArgs e)
        {
            ErrorMessage.Text = "";
            await SignInWindowsHelloAsync();
        }

        private void RegisterButtonTextBlock_OnPointerPressed(object sender, PointerRoutedEventArgs e)
        {
            ErrorMessage.Text = "";
        }
    }
}
```

```

private async Task SignInWindowsHelloAsync()
{
    if
(AccountHelper.ValidateAccountCredentials(UsernameTextBox.Text))
    {
        // Create and add a new local account
        _account =
AccountHelper.AddAccount(UsernameTextBox.Text);
        Debug.WriteLine("Successfully signed in with
traditional credentials and created local account instance!");

        //if (await
WindowsHelloHelper.CreateWindowsHelloKeyAsync(UsernameTextBox.Text))
        //{
            //    Debug.WriteLine("Successfully signed in with
Windows Hello!");
        //}
    }
    else
    {
        ErrorMessage.Text = "Invalid Credentials";
    }
}
}
}

```

- You may have noticed the commented code that was referencing a method in **WindowsHelloHelper**. In **WindowsHelloHelper.cs**, add a new method named **CreateWindowsHelloKeyAsync**. This method uses the Windows Hello API in the [KeyCredentialManager](#). Calling [RequestCreateAsync](#) will create a Windows Hello key that is specific to the *accountId* and the local machine. Please note the comments in the switch statement if you are interested in implementing this in a real world scenario.

cs

```

/// <summary>
/// Creates a Windows Hello key on the machine using the account ID
provided.
/// </summary>
/// <param name="accountId">The account ID associated with the account
that we are enrolling into Windows Hello</param>
/// <returns>Boolean indicating if creating the Windows Hello key
succeeded</returns>
public static async Task<bool> CreateWindowsHelloKeyAsync(string
accountId)
{
    KeyCredentialRetrievalResult keyCreationResult = await
KeyCredentialManager.RequestCreateAsync(accountId,
KeyCredentialCreationOption.ReplaceExisting);
}

```

```

switch (keyCreationResult.Status)
{
    case KeyCredentialStatus.Success:
        Debug.WriteLine("Successfully created key");

        // In the real world, authentication would take place on a
        // server.
        // So, every time a user migrates or creates a new Windows
        // account, details should be pushed to the server.
        // The details that would be pushed to the server include:
        // The public key, keyAttestation (if available),
        // certificate chain for attestation endorsement key (if
        // available),
        // status code of key attestation result:
        // keyAttestationCanBeRetrievedLater and
        // keyAttestationRetryType.
        // As this sample has no concept of a server, it will be
        // skipped for now.
        // For information on how to do this, refer to the second
        // sample.

        // For this sample, just return true
        return true;
    case KeyCredentialStatus.UserCanceled:
        Debug.WriteLine("User cancelled sign-in process.");
        break;
    case KeyCredentialStatus.NotFound:
        // User needs to set up Windows Hello
        Debug.WriteLine("Windows Hello is not set up!\nPlease go to
        Windows Settings and set up a PIN to use it.");
        break;
    default:
        break;
}

return false;
}

```

- Now that you have created the **CreateWindowsHelloKeyAsync** method, return to the Login.xaml.cs file and uncomment the code inside the **SignInWindowsHelloAsync** method.

```

cs

private async void SignInWindowsHelloAsync()
{
    if (AccountHelper.ValidateAccountCredentials(UsernameTextBox.Text))
    {
        //Create and add a new local account
        _account = AccountHelper.AddAccount(UsernameTextBox.Text);
        Debug.WriteLine("Successfully signed in with traditional

```

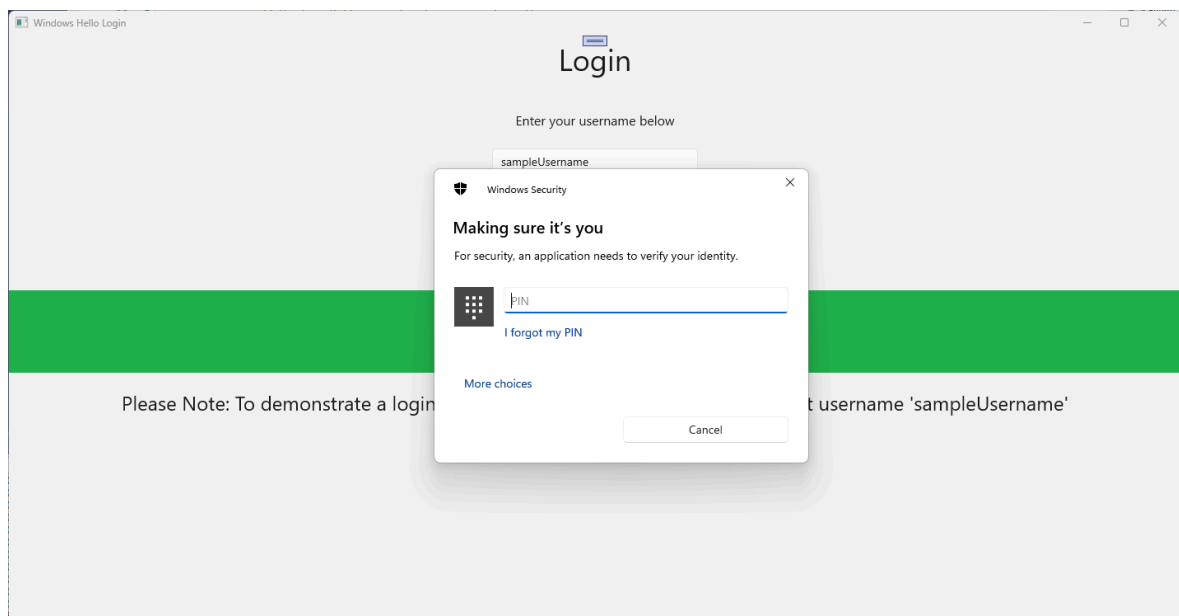
```

credentials and created local account instance!");

        if (await
WindowsHelloHelper.CreateWindowsHelloKeyAsync(UsernameTextBox.Text))
        {
            Debug.WriteLine("Successfully signed in with Windows
Hello!");
        }
        else
        {
            ErrorMessage.Text = "Invalid Credentials";
        }
    }
}

```

- Build and run the application. You will be taken to the Login page. Enter the user name as "sampleUsername" and click login. You will be prompted with a Windows Hello prompt asking you to enter your PIN. Upon entering your PIN correctly, the **CreateWindowsHelloKeyAsync** method will be able to create a Windows Hello key. Monitor the output windows to see if the messages indicating success are shown.



## Exercise 2: Welcome and User Selection Pages

In this exercise, you will continue from the previous exercise. When a user successfully logs in, they should be taken to a welcome page where they can sign out or delete their account. As Windows Hello creates a key for every machine, a user selection screen can be created, which displays all users that have been signed in on that machine. A user can then select one of these accounts and go directly to the welcome screen without needed to re-enter a password as they have already authenticated to access the machine.

- In the **Views** folder add a new blank page named "Welcome.xaml". Add the following XAML to complete the user interface for the page. This will display a title, the logged in username, and two buttons. One of the buttons will navigate back to a user list (that you will create later), and the other button will handle forgetting this user.

XML

```
<Grid>
    <StackPanel>
        <TextBlock x:Name="Title" Text="Welcome" FontSize="40"
TextAlignment="Center"/>
        <TextBlock x:Name="UserNameText" FontSize="28"
TextAlignment="Center"/>

        <Button x:Name="BackToUserListButton" Content="Back to User List"
Click="Button_Restart_Click"
            HorizontalAlignment="Center" Margin="0,20"
Foreground="White" Background="DodgerBlue"/>

        <Button x:Name="ForgetButton" Content="Forget Me"
Click="Button_Forget_User_Click"
            Foreground="White"
            Background="Gray"
            HorizontalAlignment="Center"/>
    </StackPanel>
</Grid>
```

- In the Welcome.xaml.cs code-behind file, add a new private variable that will hold the account that is logged in. You will need to implement a method to override the `OnNavigateTo` event, this will store the account passed to the **Welcome** page. You will also need to implement the `click` event for the two buttons defined in the XAML. You will need to add using statements for the `WindowsHelloLogin.Models` and `WindowsHelloLogin.Utils` namespaces.

CS

```
using WindowsHelloLogin.Models;
using WindowsHelloLogin.Utils;
using System.Diagnostics;

namespace WindowsHelloLogin.Views
{
    public sealed partial class Welcome : Page
    {
        private Account _activeAccount;

        public Welcome()
        {

```

```

        InitializeComponent();
    }

    protected override void OnNavigatedTo(NavigationEventArgs e)
    {
        _activeAccount = (Account)e.Parameter;
        if (_activeAccount != null)
        {
            UserNameText.Text = _activeAccount.Username;
        }
    }

    private void Button_Restart_Click(object sender,
RoutedEventArgs e)
    {
    }

    private void Button_Forget_User_Click(object sender,
RoutedEventArgs e)
    {
        // Remove the account from Windows Hello
        //
        WindowsHelloHelper.RemoveWindowsHelloAccountAsync(_activeAccount);

        // Remove it from the local accounts list and re-save the
        updated list
        AccountHelper.RemoveAccount(_activeAccount);

        Debug.WriteLine($"User {_activeAccount.Username}
deleted.");
    }
}

```

- You may have noticed a line commented out in the `Button_Forget_User_Click` event handler. The account is being removed from your local list but currently there is no way to be removed from Windows Hello. You need to implement a new method in `WindowsHelloHelper.cs` that will handle removing a Windows Hello user. This method will use other Windows Hello APIs to open and delete the account. In the real world, when you delete an account the server or database should be notified so the user database remains valid. You will need a `using` statement referencing the `WindowsHelloLogin.Models` namespace.

CS

```

using WindowsHelloLogin.Models;

/// <summary>
/// Function to be called when user requests deleting their account.
/// Checks the KeyCredentialManager to see if there is a Windows Hello
/// account for the current user.

```

```

/// It then deletes the local key associated with the account.
/// </summary>
public static async void RemoveWindowsHelloAccountAsync(Account
account)
{
    // Open the account with Windows Hello
    KeyCredentialRetrievalResult keyOpenResult = await
KeyCredentialManager.OpenAsync(account.Username);

    if (keyOpenResult.Status == KeyCredentialStatus.Success)
    {
        // In the real world you would send key information to server
to unregister
        //for example, RemoveWindowsHelloAccountOnServer(account);
    }

    // Then delete the account from the machine's list of Windows Hello
accounts
    await KeyCredentialManager.DeleteAsync(account.Username);
}

```

- Back in Welcome.xaml.cs, uncomment the line that calls **RemoveWindowsHelloAccountAsync**.

CS

```

private void Button_Forget_User_Click(object sender, RoutedEventArgs e)
{
    // Remove it from Windows Hello
    WindowsHelloHelper.RemoveWindowsHelloAccountAsync(_activeAccount);

    // Remove it from the local accounts list and re-save the updated
list
    AccountHelper.RemoveAccount(_activeAccount);

    Debug.WriteLine($"User {_activeAccount.Username} deleted.");
}

```

- In the **SignInWindowsHelloAsync** method (in Login.xaml.cs), once the **CreateWindowsHelloKeyAsync** is successful, it should navigate to the **Welcome** page and pass the **Account**.

CS

```

private async void SignInWindowsHelloAsync()
{
    if (AccountHelper.ValidateAccountCredentials(UsernameTextBox.Text))
    {
        // Create and add a new local account
        _account = AccountHelper.AddAccount(UsernameTextBox.Text);
        Debug.WriteLine("Successfully signed in with traditional

```

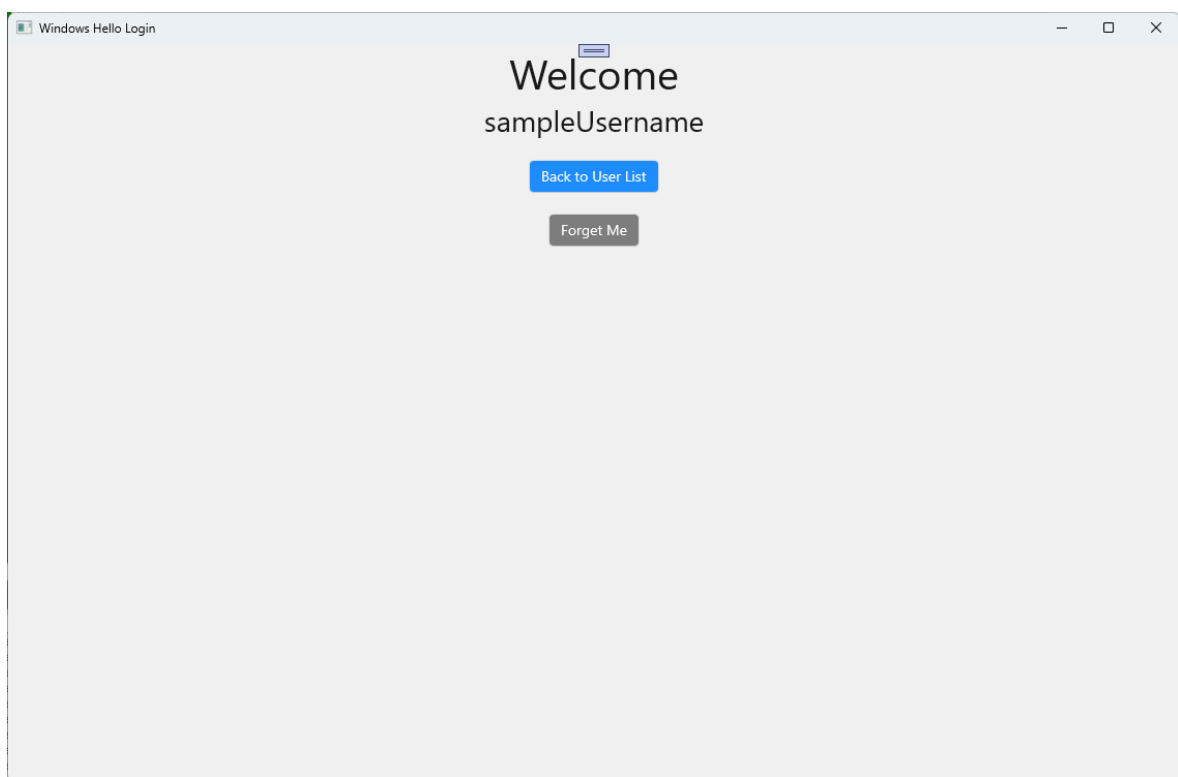
```

credentials and created local account instance!");

        if (await
WindowsHelloHelper.CreateWindowsHelloKeyAsync(UsernameTextBox.Text))
        {
            Debug.WriteLine("Successfully signed in with Windows
Hello!");
            Frame.Navigate(typeof(Welcome), _account);
        }
        else
        {
            ErrorMessage.Text = "Invalid Credentials";
        }
    }
}

```

- Build and run the application. Login with "sampleUsername" and click **Login**. Enter your PIN and if successful you should be navigated to the **Welcome** screen. Try clicking **Forget User** and monitor Visual Studio's **Output** window to see if the user was deleted. Notice that when the user is deleted, you remain on the **Welcome** page. You'll need to create a user selection page to which the app can navigate.



- In the **Views** folder, create a new blank page named "UserSelection.xaml" and add the following XAML to define the user interface. This page will contain a **ListView** that displays all the users in the local accounts list, and a **Button** that will navigate to the **Login** page to allow the user to add another account.

XML

```

<Grid>
    <StackPanel>
        <TextBlock x:Name="Title" Text="Select a User" FontSize="36"
Margin="4" TextAlignment="Center" HorizontalAlignment="Center"/>

        <ListView x:Name="UserListView" Margin="4" MaxHeight="200"
MinWidth="250" Width="250" HorizontalAlignment="Center">
            <ListView.ItemTemplate>
                <DataTemplate>
                    <Grid Background="DodgerBlue" Height="50" Width="250"
HorizontalAlignment="Stretch" VerticalAlignment="Stretch">
                        <TextBlock Text="{Binding Username}"
HorizontalAlignment="Center" TextAlignment="Center"
VerticalAlignment="Center" Foreground="White"/>
                    </Grid>
                </DataTemplate>
            </ListView.ItemTemplate>
        </ListView>

        <Button x:Name="AddUserButton" Content="+" FontSize="36" Width="60"
Click="AddUserButton_Click" HorizontalAlignment="Center"/>
    </StackPanel>
</Grid>

```

- In UserSelection.xaml.cs, implement the `Loaded` method that will navigate to the **Login** page if there are no accounts in the local list. Also implement the `SelectionChanged` event for the `ListView` and a `Click` event for the `Button`.

CS

```

using System.Diagnostics;
using WindowsHelloLogin.Models;
using WindowsHelloLogin.Utills;

namespace WindowsHelloLogin.Views
{
    public sealed partial class UserSelection : Page
    {
        public UserSelection()
        {
            InitializeComponent();
            Loaded += UserSelection_Loaded;
        }

        private void UserSelection_Loaded(object sender,
RoutedEventArgs e)
        {
            if (AccountHelper.AccountList.Count == 0)
            {
                // If there are no accounts, navigate to the Login page
                Frame.Navigate(typeof(Login));
            }
        }
    }
}

```